

ANALYSE DU STOCK ET DES VENTES DU SITE BOTTLENECK

OBJECTIF DE CE NOTEBOOK

Bienvenue dans l'outil plébiscité par les analystes de données Jupyter.

Il s'agit d'un outil permettant de mixer et d'alterner code, texte et graphiques.

Cet outil est formidable pour plusieurs raisons:

- Il permet de tester des lignes de codes au fur et à mesure de votre rédaction, de constater immédiatement le résultat d'une instruction, de la corriger si nécessaire.
- Il permet aussi de rédiger du texte pour expliquer l'approche suivie ou les résultats d'une analyse et de le mettre en forme grâce à du code html ou plus simple avec **Markdown**
- Il est possible d'ajouter des graphiques

Pour vous aider dans vos premiers pas à l'usage de Jupyter et de Python, nous avons rédigé ce notebook en vous indiquant les instructions à suivre.

Il vous suffit pour cela de saisir le code Python répondant à l'instruction donnée.

Vous verrez de temps à autre le code Python répondant à une instruction donnée mais cela est fait pour vous aider à comprendre la nature du travail qui vous est demandé.

Et gardez à l'esprit qu'il n'y a pas de solution unique pour résoudre un problème et qu'il y a autant de résolutions de problèmes que de développeurs ;)...

Etape 1 - Importation des librairies et chargement des fichiers

1.1 - Importation des librairies

```
In [110... #Permet d'afficher Les graphiques en tant qu'image et d'être exportés en pdf sans problème.
!pip uninstall -y kaleido
!pip install -q kaleido==0.2.1
```

```
Found existing installation: kaleido 0.2.1
Uninstalling kaleido-0.2.1:
  Successfully uninstalled kaleido-0.2.1
```

```
In [111... import plotly.io as pio
pio.renderers.default = "png"
```

```
In [112... #Importation de La Librairie Pandas
import pandas as pd
```

```
In [113... #Importation de La Librairie plotly express
import seaborn as sns
import matplotlib.pyplot as plt

#matplotlib.pyplot est nécessaire techniquement pour afficher une heatmap Seaborn afin de gérer :
# La taille
# Le titre
# L'affichage
```

```
In [114... #Trouver dans Google l'instruction permettant d'afficher toutes les colonnes d'un dataframe
#Saisir dans Google les mots clés "display all columns dataframe Pandas" par exemple.
#Dans les résultats de la recherche, privilégier les solutions provenant de Stack Overflow ou Medium

pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
pd.set_option('display.max_colwidth', None)
```

1.2 - Chargements des fichiers

```
In [115... #Importation du fichier web.xlsx
df_web = pd.read_excel("/content/web.xlsx")
#Importation du fichier erp.xlsx
df_erp = pd.read_excel("/content/erp.xlsx")
#Importation du fichier liaison.xlsx
df_liaison = pd.read_excel("/content/liaison.xlsx")
```

```
/usr/local/lib/python3.12/dist-packages/openpyxl/worksheet/_reader.py:329: UserWarning:
Unknown extension is not supported and will be removed

/usr/local/lib/python3.12/dist-packages/openpyxl/worksheet/_reader.py:329: UserWarning:
Unknown extension is not supported and will be removed

/usr/local/lib/python3.12/dist-packages/openpyxl/worksheet/_reader.py:329: UserWarning:
Unknown extension is not supported and will be removed
```

Etape 2 - Analyse exploratoire des fichiers

2.1 - Analyse exploratoire du fichier erp.xlsx

```
In [116... #Afficher Les dimensions du dataset
print("> Le tableau comporte {} observation(s) ou article(s)".format(df_erp.shape[0]))
print("> Le tableau comporte {} colonne(s)".format(df_erp.shape[1]))
```

→ Le tableau comporte 825 observation(s) ou article(s)
 → Le tableau comporte 6 colonne(s)

```
In [117... # Consulter Le nombre de colonnes
print("Nombre de colonnes :", df_erp.shape[1])
print('\n')

# La nature des données dans chacune des colonnes
# Le nombre de valeurs présentes dans chacune des colonnes

df_erp.info()
```

Nombre de colonnes : 6

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 825 entries, 0 to 824
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   product_id      825 non-null   int64
1   onsale_web      825 non-null   int64
2   price           825 non-null   float64
3   stock_quantity  825 non-null   int64
4   stock_status    825 non-null   object
5   purchase_price  825 non-null   float64
dtypes: float64(2), int64(3), object(1)
memory usage: 38.8+ KB
```

```
In [118... #Afficher Les 5 premières lignes de la table
df_erp.head()
```

```
Out[118... product_id  onsale_web  price  stock_quantity  stock_status  purchase_price
0         3847         1   24.2             16      instock         12.88
1         3849         1   34.3             10      instock         17.54
2         3850         1   20.8              0  outofstock         10.64
3         4032         1   14.1             26      instock          6.92
4         4039         1   46.0              3  outofstock         23.77
```

```
In [119... #Vérifier si il y a des lignes en doublon dans la colonne product_id
df_erp["product_id"].duplicated().any()
if df_erp["product_id"].duplicated().any() == False :
    print("aucun doublon")
else :
    print("il y a des doublons")
```

aucun doublon

```
In [120... #Afficher les valeurs distinctes de la colonne stock_status
df_erp["stock_status"].unique()
```

```
Out[120... array(['instock', 'outofstock'], dtype=object)
```

```
In [121... #À quelle(s) autre(s) colonne(s) sont-elles liées ?
df_erp[["product_id", "onsale_web", "stock_quantity"]].head(5)
```

```
Out[121...
  product_id  onsale_web  stock_quantity
0         3847          1              16
1         3849          1              10
2         3850          1               0
3         4032          1              26
4         4039          1               3
```

```
In [122... print("Incohérences entre les stocks et les statut")
print('_____ \n')
df_erp.loc[
    (
        (df_erp["stock_quantity"] <= 0) &
        (df_erp["stock_status"] != "outofstock")
    ) |
    (
        (df_erp["stock_quantity"] > 0) &
        (df_erp["stock_status"] != "instock")
    ),
    ["product_id", "stock_quantity", "stock_status"]
]
```

Incohérences entre les stocks et les statut

```
Out[122...
  product_id  stock_quantity  stock_status
4         4039              3    outofstock
398        4885              0      instock
```

```
In [123... print("produits proposés à la ventes mais en rupture")
print('_____ \n')
df_erp.loc[
    (df_erp["onsale_web"] == 1) &
    (df_erp["stock_quantity"] <= 0 )
]
```

produits proposés à la ventes mais en rupture

Out[123...

	product_id	onsale_web	price	stock_quantity	stock_status	purchase_price	
	2	3850	1	20.80	0	outofstock	10.64
	8	4043	1	60.00	0	outofstock	29.45
	11	4047	1	18.30	0	outofstock	9.93
	15	4051	1	7.70	0	outofstock	4.14
	16	4052	1	33.70	0	outofstock	18.11
	28	4065	1	19.50	0	outofstock	9.67
	42	4079	1	37.00	0	outofstock	19.50
	57	4100	1	15.80	0	outofstock	8.57
	70	4138	1	25.70	0	outofstock	13.01
	106	4179	1	24.00	0	outofstock	13.02
	120	4196	1	27.20	0	outofstock	14.05
	122	4198	1	5.80	0	outofstock	2.97
	155	4241	1	8.90	0	outofstock	4.37
	165	4256	1	24.40	0	outofstock	12.61
	180	4274	1	12.90	0	outofstock	6.33
	188	4283	1	27.00	0	outofstock	13.25
	208	4352	1	225.00	0	outofstock	137.81
	221	4396	1	62.00	0	outofstock	28.42
	242	4594	1	144.00	0	outofstock	87.36
	253	4606	1	50.10	0	outofstock	24.59
	258	4612	1	20.60	0	outofstock	10.22
	264	4618	1	30.60	0	outofstock	16.44
	279	4636	1	50.00	0	outofstock	25.58
	326	4706	1	16.80	0	outofstock	8.68
	344	4726	1	12.70	0	outofstock	6.82
	386	4858	1	6.50	0	outofstock	3.19
	395	4870	1	9.30	0	outofstock	4.81
	397	4876	1	22.80	0	outofstock	11.90
	398	4885	1	18.70	0	instock	9.66
	426	4923	1	7.00	0	outofstock	3.65
	452	4976	1	16.45	0	outofstock	8.07
	486	5070	1	84.70	0	outofstock	47.43
	487	5075	1	43.30	0	outofstock	21.70
	494	5384	1	28.80	0	outofstock	15.18
	519	5487	1	43.50	0	outofstock	23.37
	531	5544	1	61.60	0	outofstock	31.51
	545	5565	1	92.00	0	outofstock	46.11
	561	5617	1	27.80	0	outofstock	14.65
	573	5700	1	44.50	-1	outofstock	22.30
	578	5707	1	10.80	0	outofstock	5.69
	579	5709	1	31.70	0	outofstock	16.54
	581	5712	1	57.60	0	outofstock	30.36
	655	5930	1	14.10	0	outofstock	6.92
	675	6038	1	48.50	0	outofstock	25.31
	680	6050	1	38.50	0	outofstock	20.09
	722	6299	1	78.00	0	outofstock	41.51
	723	6301	1	40.50	0	outofstock	20.51

	product_id	onsale_web	price	stock_quantity	stock_status	purchase_price
752	6632	1	52.70	0	outofstock	26.41

```
In [124... print("produits non proposés à la ventes mais en stock")
print('_____ \n')
df_erp.loc[
    (df_erp["onsale_web"] == 0) &
    (df_erp["stock_quantity"] > 0 )
]
```

produits non proposés à la ventes mais en stock

```
Out[124...
```

	product_id	onsale_web	price	stock_quantity	stock_status	purchase_price
123	4200	0	5.8	33	instock	3.12
234	4565	0	30.5	3	instock	15.92
238	4577	0	49.0	1	instock	24.05
239	4578	0	40.0	3	instock	20.05
318	4692	0	12.0	48	instock	6.39
...	...	...	...	...	...	...
818	7200	0	31.0	6	instock	15.54
819	7201	0	31.0	18	instock	16.02
820	7203	0	45.0	30	instock	23.48
821	7204	0	45.0	9	instock	24.18
823	7329	0	26.5	14	instock	13.42

65 rows × 6 columns

```
In [125... #Création d'une colonne "stock_status_2"
#La valeur de cette deuxième colonne sera fonction de la valeur dans la colonne "stock_quantity"
#Si la valeur de la colonne "stock_quantity" est nulle, renseigner "outofstock" sinon mettre "instock"
df_erp["stock_status_2"] = None
df_erp.loc[df_erp["stock_quantity"] > 0, "stock_status_2"] = "instock"
df_erp.loc[df_erp["stock_quantity"] <= 0, "stock_status_2"] = "outofstock"
df_erp
```

```
Out[125...
```

	product_id	onsale_web	price	stock_quantity	stock_status	purchase_price	stock_status_2
0	3847	1	24.2	16	instock	12.88	instock
1	3849	1	34.3	10	instock	17.54	instock
2	3850	1	20.8	0	outofstock	10.64	outofstock
3	4032	1	14.1	26	instock	6.92	instock
4	4039	1	46.0	3	outofstock	23.77	instock
...	...	...	...	...	...	...	...
820	7203	0	45.0	30	instock	23.48	instock
821	7204	0	45.0	9	instock	24.18	instock
822	7247	1	54.8	6	instock	27.18	instock
823	7329	0	26.5	14	instock	13.42	instock
824	7338	1	16.3	40	instock	8.00	instock

825 rows × 7 columns

```
In [126... #Vérifions que les 2 colonnes sont identiques:
#Les 2 colonnes sont strictement identiques si les valeurs de chaque ligne sont strictement identiques 2 à 2
#La comparaison de 2 colonnes peut se réaliser simplement avec l'instruction ci-dessous:
df_erp[df_erp["stock_status_2"] != df_erp["stock_status"]]

# note Thomas : j'utilise l'attribut loc pour plus de lisibilité et comme bonne pratique → syntaxe : loc(lignes,colonnes)
df_erp.loc[
    df_erp["stock_status_2"] != df_erp["stock_status"], #lignes à retenir
    ["product_id", "stock_status", "stock_status_2"] #colonnes à afficher
] #fin de l'attribut
```

*#Le résultat est l'affichage de True ou False pour chacune des lignes du dataset
#C'est un bon début, mais difficile à exploiter*

```
Out[126...

|     | product_id | stock_status | stock_status_2 |
|-----|------------|--------------|----------------|
| 4   | 4039       | outofstock   | instock        |
| 398 | 4885       | instock      | outofstock     |


```

```
In [127...#Mais il est possible de synthétiser ce résultat en effectuant la somme de cette colonne:  
#True vaut 1 et False 0  
#Nous devrions obtenir la somme de 823 qui correspond au nombre de lignes dans ce dataset  
tableau = {  
    "statuts conformes" : int((df_erp["stock_status_2"] == df_erp["stock_status"]).sum()),  
    "statuts non conformes" : int((df_erp["stock_status_2"] != df_erp["stock_status"]).sum())  
}  
  
tableau
```

```
Out[127... {'statuts conformes': 823, 'statuts non conformes': 2}
```

```
In [128...#Si les colonnes ne sont absolument pas identiques ligne à ligne alors identifier la ligne en écart  
##Dans ce cas je vous donne ce lien pour apprendre à réaliser des filtres dans Pandas:  
##https://bitbucket.org/hrojas/Learn-pandas/src/master/  
##Lesson 3  
df_erp[df_erp["stock_status"] != df_erp["stock_status_2"]]
```

```
Out[128...

|     | product_id | onsale_web | price | stock_quantity | stock_status | purchase_price | stock_status_2 |
|-----|------------|------------|-------|----------------|--------------|----------------|----------------|
| 4   | 4039       | 1          | 46.0  | 3              | outofstock   | 23.77          | instock        |
| 398 | 4885       | 1          | 18.7  | 0              | instock      | 9.66           | outofstock     |


```

```
In [129...#Corriger la ou les données incohérentes  
df_erp.loc[  
    df_erp["stock_status"] != df_erp["stock_status_2"],  
    "stock_status"  
] = df_erp["stock_status_2"]  
  
#Vérification en utilisant le même code que plus haut pour afficher les problèmes  
nb_erreur = int((df_erp["stock_status"] != df_erp["stock_status_2"]).sum())  
print('il y a actuellement', nb_erreur, ' erreur(s)')
```

il y a actuellement 0 erreur(s)

2.1.1 - Analyse exploratoire de chaque variable du fichier erp.xlsx

2.1.1.1 - Analyse de la variable PRIX

```
In [130...#####  
## LES PRIX ##  
#####  
  
#Vérification des prix: Y a-t-il des prix non renseignés, négatifs ou nuls?  
  
tableau2 = pd.DataFrame(  
    {  
        "Anomalies sur les prix" : ["Prix non renseignés", "Prix nuls ou négatifs"],  
        "Nombre de lignes" : [  
            df_erp["price"].isna().sum(), (df_erp["price"] <= 0).sum()  
        ]  
    }  
)  
  
tableau2
```

```
Out[130...

|   | Anomalies sur les prix | Nombre de lignes |
|---|------------------------|------------------|
| 0 | Prix non renseignés    | 0                |
| 1 | Prix nuls ou négatifs  | 3                |


```

```
In [131...#Afficher le ou les prix non renseignés dans la colonne "price"  
#Saisir l'instruction manquante dans la fonction format  
print(  
    "Nombres d'articles avec un prix non renseigné: {}".format(  
        df_erp["price"].isna().sum()  
    )
```

```
)

df_erp.loc[
    df_erp["price"].isna(),
    ["product_id", "price"]
]
```

Nombres d'articles avec un prix non renseigné: 0

Out[131... **product_id price**

In [132... *#Afficher le prix minimum de la colonne "price"*
df_erp["price"].min()

Out[132... -20.0

In [133... *#Afficher le prix maximum de la colonne "price"*
df_erp["price"].max()

Out[133... 225.0

In [134... *#Afficher les prix inférieurs à 0*
df_erp.loc[
 df_erp["price"] < 0, ["product_id", "price"]
]

Out[134... **product_id price**

151	4233	-20.0
469	5017	-8.0
739	6594	-9.1

In [135... *#qu'est-ce qu'il faut en faire ?*

```
print("il faut corriger ces prix et les retirer immédiatement de la vente si elles sont proposées à la vente")

# code -> df_erp.loc[df_erp["price"] <= 0, "onsale_web"] = 0
# néanmoins les lignes concernées ont déjà pour valeurs onsale_web = 0
```

il faut corriger ces prix et les retirer immédiatement de la vente si elles sont proposées à la vente

2.1.1.2 - Analyse de la variable STOCK

In [136... #####
stock_quantity ###
#####

```
#Vérification de la colonne stock quantity
#Afficher la quantité minimum de la colonne "stock_quantity"
df_erp["stock_quantity"].min()
```

Out[136... -10

In [137... *#Afficher la quantité maximum de la colonne "stock_quantity"*
df_erp["stock_quantity"].max()

Out[137... 145

In [138... *#Afficher les stocks inférieurs à 0*
df_erp.loc[
 df_erp["stock_quantity"] < 0, ["product_id", "stock_quantity"]
]

Out[138... **product_id stock_quantity**

449	4973	-10
573	5700	-1

In [139... *#(qu'est-ce qu'il faut en faire ?)*
print("Vérifier le stock réel pour ces articles et faire la correction informatique. Chercher également d'où provient l'erreur")

Vérifier le stock réel pour ces articles et faire la correction informatique. Chercher également d'où provient l'erreur.

2.1.1.3 - Analyse de la variable ONSALE_WEB

```
In [140... #Vérification de la colonne onsale_web et des valeurs qu'elle contient.
df_erp["onsale_web"].unique()
```

```
Out[140... array([1, 0])
```

```
In [141... #Que signifient-elles ?
signification = pd.DataFrame(
    {
        "Signification" : ["produit en vente via le site web","produit non vendu sur le site web"],
        "Valeur" : [1,0]
    }
)

signification
```

```
Out[141...      Signification  Valeur
0   produit en vente via le site web    1
1   produit non vendu sur le site web    0
```

```
In [142... #Quelles sont Les colonnes à conserver selon vous?
```

```
In [143... #Supprimer la colonne comportant le libellé "stock_status_2" car elle est redondante
#avec la colonne "stock_status".
df_erp = df_erp.drop(columns=["stock_status_2"])
df_erp
```

```
Out[143...      product_id  onsale_web  price  stock_quantity  stock_status  purchase_price
0             3847         1   24.2             16      instock         12.88
1             3849         1   34.3             10      instock         17.54
2             3850         1   20.8              0  outofstock         10.64
3             4032         1   14.1             26      instock          6.92
4             4039         1   46.0              3      instock         23.77
...           ...         ...   ...             ...           ...           ...
820            7203         0   45.0             30      instock         23.48
821            7204         0   45.0              9      instock         24.18
822            7247         1   54.8              6      instock         27.18
823            7329         0   26.5             14      instock         13.42
824            7338         1   16.3             40      instock          8.00
```

825 rows × 6 columns

2.1.1.4 - Analyse de la variable prix d'achat

```
In [144... #####
##  prix d'achat  ##
#####

#Vérification de la colonne purchase_price :
#Afficher le ou les prix non renseignés dans la colonne "purchase_price"
df_erp.loc[df_erp["purchase_price"].isna(),["product_id","purchase_price"]]
```

```
Out[144...      product_id  purchase_price
```

```
In [145... #Afficher le prix minimum de la colonne "purchase_price"
df_erp.loc[
    df_erp["purchase_price"] == df_erp["purchase_price"].min(),
    ["product_id","purchase_price"]
]

# Thomas : permet d'afficher les articles dans le cas où il y en aurait plusieurs au prix minimum.
```

```
Out[145...      product_id  purchase_price
700            6129          2.74
```



```
In [146... #Afficher Le prix maximum de la colonne "purchase_price"

df_erp.loc[
    df_erp["purchase_price"] == df_erp["purchase_price"].max(),
    ["product_id", "purchase_price"]
]

# Thomas : permet d'afficher les articles dans le cas où il y en aurait plusieurs au prix maximum.
```

```
Out[146...      product_id  purchase_price
208          4352          137.81
```

2.2 - Analyse exploratoire du fichier web.xlsx

```
In [147... #Dimension du dataset
#Nombre de caractéristiques
#Nombre d'observations
#Consulter le nombre de colonnes
#La nature des données dans chacune des colonnes
#Le nombre de valeurs présentes dans chacune des colonnes

df_web.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1513 entries, 0 to 1512
Data columns (total 29 columns):
#   Column                Non-Null Count  Dtype
---  -
0   sku                    1428 non-null   object
1   virtual                1513 non-null   int64
2   downloadable           1513 non-null   int64
3   rating_count           1513 non-null   int64
4   average_rating         1430 non-null   float64
5   total_sales            1430 non-null   float64
6   tax_status             716 non-null    object
7   tax_class              0 non-null      float64
8   post_author            1430 non-null   float64
9   post_date              1430 non-null   datetime64[ns]
10  post_date_gmt           1430 non-null   datetime64[ns]
11  post_content            0 non-null      float64
12  product_type            1429 non-null   object
13  post_title              1430 non-null   object
14  post_excerpt            716 non-null    object
15  post_status             1430 non-null   object
16  comment_status          1430 non-null   object
17  ping_status             1430 non-null   object
18  post_password           0 non-null      float64
19  post_name               1430 non-null   object
20  post_modified            1430 non-null   datetime64[ns]
21  post_modified_gmt        1430 non-null   datetime64[ns]
22  post_content_filtered    0 non-null      float64
23  post_parent             1430 non-null   float64
24  guid                    1430 non-null   object
25  menu_order              1430 non-null   float64
26  post_type               1430 non-null   object
27  post_mime_type           714 non-null    object
28  comment_count            1430 non-null   float64
dtypes: datetime64[ns](4), float64(10), int64(3), object(12)
memory usage: 342.9+ KB
```

```
In [148... #Selon vous, quelles sont les colonnes à conserver ?
#Si vous avez défini des colonnes à supprimer, effectuer l'opération
liste_des_colonnes_a_supprimer = ["tax_class", "post_content", "post_password", "post_content_filtered", "virtual", "downloadable"]
for col in liste_des_colonnes_a_supprimer:
    df_web.drop(col, axis=1, inplace=True)
```

```
In [148... 
```

```
In [149... #Visualisation des valeurs de la colonne sku

df_web["sku"].unique()
```

```

Out[149... array([11862, 16057, 14692, 16295, 15328, 15471, 16515, 16246, nan, 13572,
16513, 16585, 16269, 15526, 12869, 15575, 11586, 14338, 15425,
16560, 15361, 13809, 11587, 15022, 14323, 16342, 16029, 15475,
13754, 14680, 15875, 9636, 13849, 13662, 16564, 13557, 15429,
14712, 15032, 15481, 15448, 16580, 15441, 804, 15300, 13958, 16071,
15678, 13895, 15711, 12882, 16053, 13766, 16247, 12640, 15292,
15476, 15670, 16189, 16038, 14864, 16044, 15324, 15531, 15953,
15413, 15733, 14366, 15895, 15892, 16472, 15185, 16010, 15793,
15849, 12315, 15741, 15934, 15148, 15781, 15659, 15106, 15490,
14507, 14149, 16307, 13736, 14090, 16037, 15758, 14661, 12587,
15337, 15489, 15201, 16305, 16131, 13435, 15747, 12203, 14509,
14768, 16262, 14561, 16505, 15717, 16129, 13460, 15871, 15940,
11602, 13127, 13520, 15480, 13032, 15436, 15269, 15910, 19821,
16263, 15138, 15146, 15126, 15482, 16186, 13905, 16540, 15856,
15677, 14700, 15325, 19815, 3506, 16056, 14975, 15341, 15204,
15415, 16065, 15479, 16151, 15127, 15140, 15779, 15473, 15530,
14805, 14106, 9937, 15281, 16553, 15315, 15668, 13211, 15161,
11258, 16296, 12588, 15792, 15921, 15690, 15775, 15577, 15870, 802,
15163, 15707, 15561, 15539, 4679, 16320, 15923, 3509, 2534, 16238,
14819, 15705, 15254, 15554, 14451, 13215, 14302, 12476, 16274,
14696, 14573, 15343, 791, 14474, 12586, 15346, 15834, 15342, 15141,
15038, 14265, 15238, 16586, 13073, 15664, 14527, 14845, 13172,
13996, 15864, 15440, 15759, 15706, 11277, 15676, 15731, 16525,
15785, 14699, 16063, 15718, 14372, 15881, 793, 14775, 12771, 15426,
13291, 16023, 15713, 15675, 15282, 15927, 15351, 12791, '13127-1',
13853, 15613, 11849, 15478, 16121, 16237, 15461, 15745, 15621,
15683, 12639, 15466, 15184, 13078, 13904, 14828, 14000, 15612, 304,
15811, 15649, 14599, 15033, 13959, 15026, 16449, 15095, 15829,
13965, 16306, 16276, 15729, 15303, 14679, 15318, 13599, 15206,
19816, 16153, 16067, 15241, 14981, 16264, 14915, 15737, 16289,
15229, 7033, 15654, 15879, 16256, 15710, 16498, 15349, 11933,
16462, 14596, 15296, 16042, 15887, 15264, 15307, 15369, 14095,
15868, 14371, 12657, 15808, 15787, 13604, 15605, 15795, 15566,
16210, 15732, 15661, 13814, 16130, 16003, 14676, 13516, 14977,
14945, 12641, 14469, 15237, 14905, 15462, 16031, 16191, 3568,
15080, 15353, 15861, 16529, 16068, 16081, 14729, 16077, 15373,
11049, 16154, 14809, 15073, 15345, 15756, 13531, 15614, 7086,
15784, 14332, 14300, 14141, 15734, 15456, 805, 15766, 15667, 16152,
16318, 15360, 13517, 7819, 15740, 15930, 16155, 15339, 14950, 2179,
15688, 13982, 38, 15178, 16209, 14982, 8463, 13412, 15298, 15770,
15487, 15205, 13230, 14220, 15227, 16094, 15859, 15179, 14774,
14944, 16022, 15767, 15338, 13913, 14756, 15746, 16192, 15763,
12365, 9562, 16039, 16328, 15196, 531, 15810, 2361, 15402, 15945,
15807, 14506, 14100, 15075, 13765, 16093, 10814, 12942, 1366,
12857, 12045, 13515, 11601, 16148, 12339, 16567, 15240, 15414, 812,
16292, 11736, 15714, 15120, 13117, 3383, 15631, 15136, 15145,
13096, 15857, 14632, 11641, 15736, 15615, 15574, 16096, 15564,
15774, 14930, 15941, 16045, 16230, 14429, 15769, 16265, 14657,
15261, 16239, 15715, 13453, 16244, 15147, 16280, 14508, 15465,
3510, 14374, 15880, 13416, 15183, 15860, 11467, 11585, 14485,
16138, 19822, 15155, 11996, 14626, 16330, 15035, 15404, 8365, 3507,
14773, 15967, 15812, 16319, 16004, 15310, 16097, 14856, 15629,
1364, 15116, 14101, 14912, 15036, 16047, 14184, 14983, 16030,
16283, 16324, 15801, 10014, 12881, 15660, 16005, 15306, 16277,
15527, 15428, 13957, 15647, 12496, 15399, 13969, 15525, 13914,
12790, 15720, 13313, 15928, 15432, 15655, 16539, 12589, 15648,
16011, 16146, 15672, 15030, 15735, 14800, 16190, 13217, 13854,
16317, 16565, 15891, 15730, 16497, 16124, 10459, 19814, 15863,
15786, 15791, 16135, 16147, 16066, 15818, 15790, 8193, 16255,
15047, 15966, 14736, 16120, 16013, 798, 1662, 16416, 7032, 14581,
15783, 16144, 5646, 8344, 13514, 15848, 15658, 14897, 12194, 10775,
16304, 14980, 14192, 15952, 14099, 15125, 13089, 16062, 14923,
14839, 15576, 15657, 1360, 15583, 15457, 16275,
'bon-cadeau-25-euros', 15797, 12366, 15764, 16072, 15070, 16323,
15753, 15755, 15582, 13379, 15704, 13072, 15862, 15533, 16028,
14725, 13647, 16261, 15944, 15256, 523, 15452, 16322, 16014, 14600,
15316, 15776, 16211, 14899, 14955, 13910, 15134, 15434, 15813,
15794, 15946, 16213, 14855, 14827, 14089, 15283, 14647, 16504,
15674, 16537, 41, 13762, 14570, 16041, 15202, 15180, 15375, 16046,
16034, 14092, 14241, 11997, 15922, 14844, 15444, 15486, 15378, 807,
13567, 16281, 15213, 16166, 11225, 15773, 16160, 19820, 7818,
14751, 15280, 15933, 13627, 15403, 14580, 16024, 14797, 12494,
16229, 13659, 16578, 14802, 15850, 16159, 15839, 13052, 15869,
14604, 16273, 15329, 14253, 14941, 15567, 16132, 14746, 15656,
16527, 15162, 16326, 16501, 16119, 15662, 15344, 6616, 15958,
14865, 11669, 14395, 15149, 12585, 11668, 11847, 12599, 16133,
15665, 15382, 13209, 14461, 15748, 15663, 15072, 15004, 16069,
16180, 15949, 16149, 16043, 15845, 15951, 13074, 14569],
dtype=object)

```

```

In [150... #Quelles sont Les valeurs qui ne semblent pas respecter la règle de codification?
#Si vous avez identifié des codes articles ne respectant pas la règle de codification, consultez-les
df_web.loc[
    (df_web["sku"].notna()) &
    (pd.to_numeric(df_web["sku"], errors="coerce").isna()),

```

```
                "sku"
            ].drop_duplicates()
```

Out[150...

	sku
272	13127-1
842	bon-cadeau-25-euros

dtype: object

```
In [151... #Identifier les lignes sans code article
df_web.loc[df_web["sku"].isnull(),"sku"]
```

Out[151...

	sku
8	NaN
20	NaN
30	NaN
37	NaN
41	NaN
...	...
1384	NaN
1429	NaN
1432	NaN
1445	NaN
1457	NaN

85 rows × 1 columns

dtype: object

```
In [152... # Mise à jour du dataframe df_web pour retirer les lignes sans "sku"
df_web["sku"].value_counts().gt(1).sum()
print("il y a ",df_web["sku"].value_counts().gt(1).sum()," sku en double")
```

il y a 714 sku en double

```
In [153... # je nettoie les lignes vides (sans "sku")
df_web = df_web[df_web["sku"].notnull()].copy()
print("Nb de lignes initial : 1513 lignes","\nNb de lignes vides : 85","\nNombre de doublons : 714","\nNb de lignes du Dat
```

Nb de lignes initial : 1513 lignes
Nb de lignes vides : 85
Nombre de doublons : 714
Nb de lignes du Dataframe au final : 714

```
In [154... #Pour les codes articles identifiés, réaliser une analyse et définir l'action à entreprendre
df_web[df_web["post_type"] == "product"]
```

	sku	total_sales	tax_status	post_author	post_date	post_date_gmt	product_type	post_title		
	2	14692	5.0	taxable	2.0	2019-03-19 10:06:47	2019-03-19 09:06:47	Vin	Château Fonréaud Bordeaux Blanc Le Cygne 2016	<div>Grâce à la complémentari le constituant, Le Cygne est un et aérien, offrant une riche fruitée et florale, une rondeur se
	4	15328	2.0	taxable	2.0	2019-03-27 18:05:09	2019-03-27 17:05:09	Vin	Agnès Levet Côte Rôtie Maestria 2017	C'est un vin élégant et d du lieu-dit La Landonn complété par des notes f soutenu par le velours des tanir
	6	16515	10.0	taxable	2.0	2018-06-02 09:31:31	2018-06-02 07:31:31	Vin	Château Turcaud Bordeaux Rouge Cuvée Majeure 2018	<div id="wrapper">\r wrapper">\n<div id="conten class="contenu"> <sect class="wysiwyg">\n\n<spa background-color: transpa fc Neue',Helvetica,Roboto,Arial 16px; font-style: normal; font-v weight: 400; letter-spacing 25.6px; text-align: left; text-de indent: 0px;"> Fruit rouge bier fumé et grillé, lié au ter barriques. <spa background-color: transpa fc Neue',Helvetica,Roboto,Arial 16px; font-style: normal; font-v weight: 400; letter-spacing 25.6px; text-align: left; text-de indent: 0px;">Riche, concentr avec de la fraîcheur. L'élevage aux tanins de bien se for rondeur.\n\n</div>\n</div>\n</div>\n</di id="footer-wrapper" container"> </div>\
	11	16585	15.0	taxable	2.0	2018-02-16 14:03:16	2018-02-16 13:03:16	Vin	Xavier Frissant Touraine Sauvignon 2019	Un joli sauvignon frais et mir notes exotiques. Des très b
	14	12869	7.0	taxable	2.0	2019-03-28 14:29:35	2019-03-28 13:29:35	Vin	Stéphane Tissot Arbois D.D. 2016	Un Vin coloré et éclatant. arômes de griotte et de p d'épices douces. La bouch disposant beaux tanins
	...	...	...	...	...	...	...	...	...	
	1503	13074	4.0	taxable	2.0	2018-02-12 14:25:28	2018-02-12 13:25:28	Vin	Château de Vaudieu Châteauneuf-du-Pape L'Avenue 2015	"L'Avenue" est issue d't grenaches sur grès au sommet c Châteauneuf-du-Pape. Le nez coulis de framboise et de thé n la fraîcheur et la finesse cara grenaches sur se
	1505	16322	0.0	taxable	2.0	2018-02-15 13:51:32	2018-02-15 12:51:32	Vin	Moulin de Gassac IGP Pays d'Hérault Guilhem Rouge 2019	Belle complexité aromatique a épices. Attaque souple, et frui fondu. Un vin f
	1507	12365	10.0	taxable	2.0	2019-01-29 15:53:05	2019-01-29 14:53:05	Vin	Parés Baltà Penedès Electio 2013	Une cuvée produite avec une Xarel·lo. Un vin délicat et con persistance aromatique. La l accompagnée de notes de fruit
	1508	16326	5.0	taxable	2.0	2019-04-18 11:32:46	2019-04-18 09:32:46	Vin	Camin Larredya Jurançon Moelleux Au Capcéu 2018	Sur le millésime 2017, Au Camin Larredya est un magnifique Jurançon de garde s notes de fruits jaunes, fruits exc L'acidité contrebalance magnific

	sku	total_sales	tax_status	post_author	post_date	post_date_gmt	product_type	post_title
	1509	15662	15.0	taxable	2.0	2018-02-27 10:13:03	2018-02-27 09:13:03	Vin
								C'est le Beaujolais typique : fr Chermette croquant avec un côté acidulé d Domaine du griottes, framboise, fra Vissoux excellence, il se boit Beaujolais décontractée. Il se marie avec u Griottes 2018 circonstances et de mets. b

714 rows × 21 columns

```
In [155... #La clé pour chaque ligne est-elle unique? autrement dit, y a-t-il des doublons?
if df_web["sku"].duplicated().any() == True : print("Oui, il y a des doublons\nCes derniers sont dus au fait que chaque ar
else : print("il n'y a pas de doublons")
```

Oui, il y a des doublons
Ces derniers sont dus au fait que chaque article a 2 lignes dans la base

```
In [156... # je supprime les lignes en doublons liées aux images ("attachment")
df_web_products = df_web[df_web["post_type"] == "product"].copy()
```

```
In [157... #Les lignes sans code article semblent être toutes non renseignées
#Pour s'en assurer, réaliser les étapes suivantes:
#1 - Créer un dataframe avec uniquement les lignes sans code article
df_web[df_web["sku"].isnull()]

#2 - Utiliser la fonction df.info() sur ce nouveau dataframe pour observer le nombre de valeurs renseignées dans chacune d
df_web[df_web["sku"].isnull()].info()

#3 - Que constatez-vous?

# 1. Les valeurs 0 sont présentes dans les colonnes 'virtual', 'downloadable', 'rating count'
# 2. 2 articles n'ont pas de SKU et affichent des ventes négatives.
# Pierre Jean Villa Condrieu Jardin Suspendu 2018
# Pierre Jean Villa Côte Rôtie Fongéant 2017
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 0 entries
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   sku                    0 non-null      object
1   total_sales            0 non-null      float64
2   tax_status             0 non-null      object
3   post_author            0 non-null      float64
4   post_date              0 non-null      datetime64[ns]
5   post_date_gmt          0 non-null      datetime64[ns]
6   product_type           0 non-null      object
7   post_title             0 non-null      object
8   post_excerpt           0 non-null      object
9   post_status            0 non-null      object
10  comment_status         0 non-null      object
11  ping_status            0 non-null      object
12  post_name              0 non-null      object
13  post_modified           0 non-null      datetime64[ns]
14  post_modified_gmt       0 non-null      datetime64[ns]
15  post_parent            0 non-null      float64
16  guid                   0 non-null      object
17  menu_order             0 non-null      float64
18  post_type              0 non-null      object
19  post_mime_type         0 non-null      object
20  comment_count          0 non-null      float64
dtypes: datetime64[ns](4), float64(5), object(12)
memory usage: 0.0+ bytes
```

In [157...

2.3 - Analyse exploratoire du fichier liaison.xlsx

```
In [158... #Dimension du dataset:

#Nombre d'observations
#Nombre de caractéristiques
print("le dataset contient ",df_liaison.shape[0]," lignes et ",df_liaison.shape[1]," colonnes")
print("il s'agit d'une table de correspondance pour faire le lien entre les données issues de l'erp et celles issues du si
```

le dataset contient 825 lignes et 2 colonnes
il s'agit d'une table de correspondance pour faire le lien entre les données issues de l'erp et celles issues du site web

```
In [159... #Consulter le nombre de colonnes
#La nature des données dans chacune des colonnes
#Le nombre de valeurs présentes dans chacune des colonnes

df_liaison.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 825 entries, 0 to 824
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  -
0   id_web       734 non-null    object
1   product_id   825 non-null    int64
dtypes: int64(1), object(1)
memory usage: 13.0+ KB
```

```
In [160... #Les valeurs de la colonne "product_id" sont-elles toutes uniques?
if df_liaison["product_id"].duplicated().any() : print("Non, il y a des doublons")
else : print("Oui, chaque valeur est unique")
```

Oui, chaque valeur est unique

```
In [161... #Les valeurs de la colonne "id_web" sont-elles toutes uniques?
if df_liaison["id_web"].dropna().is_unique == True : print("Oui, chaque valeur est unique")
else : print("Non, il y a des doublons")
```

Oui, chaque valeur est unique

```
In [162... #Avons-nous des articles sans correspondance?
if df_liaison["id_web"].isna().any() == True : print(df_liaison["id_web"].isna().sum(), " lignes n'ont pas de valeur")
else: print("non, tout va bien de ce côté-là")
```

91 lignes n'ont pas de valeur

Etape 3 - Jonction des fichiers

Etape 3.1 - Jonction du fichier df_erp et df_liaison

```
In [163... #Fusion des fichiers df_erp et df_liaison
```

```
df_merge = df_erp.merge(
    df_liaison,
    on="product_id",
    how="outer",
    indicator=True
)

df_merge
```

```
Out[163...
   product_id  onsale_web  price  stock_quantity  stock_status  purchase_price  id_web  _merge
0           0         3847      1    24.2          16      instock          12.88  15298    both
1           1         3849      1    34.3          10      instock          17.54  15296    both
2           2         3850      1    20.8           0  outofstock          10.64  15300    both
3           3         4032      1    14.1          26      instock           6.92  19814    both
4           4         4039      1    46.0           3      instock          23.77  19815    both
...         ...         ...    ...           ...           ...           ...     ...     ...
820          0         7203      0    45.0          30      instock          23.48    NaN    both
821          0         7204      0    45.0           9      instock          24.18    NaN    both
822          1         7247      1    54.8           6      instock          27.18  13127-1  both
823          0         7329      0    26.5          14      instock          13.42  14680-1  both
824          1         7338      1    16.3          40      instock           8.00  16230    both
```

825 rows × 8 columns

```
In [164... df_merge.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 825 entries, 0 to 824
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   product_id      825 non-null   int64
1   onsale_web       825 non-null   int64
2   price           825 non-null   float64
3   stock_quantity  825 non-null   int64
4   stock_status    825 non-null   object
5   purchase_price  825 non-null   float64
6   id_web          734 non-null   object
7   _merge          825 non-null   category
dtypes: category(1), float64(2), int64(3), object(2)
memory usage: 46.2+ KB
```

In [165... *#Y a t-il des lignes ne "matchant" pas entre les 2 fichiers?*

```
df_merge.loc[df_merge["_merge"] != "both"]
```

Out[165... **product_id onsale_web price stock_quantity stock_status purchase_price id_web _merge**

In [166... `df_merge.drop("_merge",axis=1,inplace=True)`

In [167... `df_merge`

Out[167...

	product_id	onsale_web	price	stock_quantity	stock_status	purchase_price	id_web
0	3847	1	24.2	16	instock	12.88	15298
1	3849	1	34.3	10	instock	17.54	15296
2	3850	1	20.8	0	outofstock	10.64	15300
3	4032	1	14.1	26	instock	6.92	19814
4	4039	1	46.0	3	instock	23.77	19815
...	...	...	...	...	...	...	...
820	7203	0	45.0	30	instock	23.48	NaN
821	7204	0	45.0	9	instock	24.18	NaN
822	7247	1	54.8	6	instock	27.18	13127-1
823	7329	0	26.5	14	instock	13.42	14680-1
824	7338	1	16.3	40	instock	8.00	16230

825 rows × 7 columns

Etape 3.2 - Jonction du fichier df_merge et df_web

In [168... *#Fusionner les datasets df_merge et df_web*

```
df_final = df_merge.merge(
    df_web_products,
    left_on="id_web",
    right_on="sku",
    how="outer",
    indicator=True)

df_base_finale = df_final.copy()

df_final.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 825 entries, 0 to 824
Data columns (total 29 columns):
#   Column                Non-Null Count  Dtype
---  -
0   product_id            825 non-null   int64
1   onsale_web            825 non-null   int64
2   price                 825 non-null   float64
3   stock_quantity        825 non-null   int64
4   stock_status          825 non-null   object
5   purchase_price        825 non-null   float64
6   id_web               734 non-null   object
7   sku                  714 non-null   object
8   total_sales          714 non-null   float64
9   tax_status           714 non-null   object
10  post_author          714 non-null   float64
11  post_date            714 non-null   datetime64[ns]
12  post_date_gmt        714 non-null   datetime64[ns]
13  product_type         713 non-null   object
14  post_title           714 non-null   object
15  post_excerpt         714 non-null   object
16  post_status          714 non-null   object
17  comment_status       714 non-null   object
18  ping_status          714 non-null   object
19  post_name            714 non-null   object
20  post_modified        714 non-null   datetime64[ns]
21  post_modified_gmt    714 non-null   datetime64[ns]
22  post_parent          714 non-null   float64
23  guid                714 non-null   object
24  menu_order           714 non-null   float64
25  post_type            714 non-null   object
26  post_mime_type       0 non-null     object
27  comment_count        714 non-null   float64
28  _merge              825 non-null   category
dtypes: category(1), datetime64[ns](4), float64(7), int64(3), object(14)
memory usage: 181.5+ KB
```

In [168...

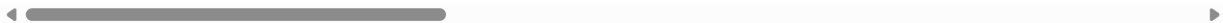
In [169...

```
df_final.loc[
    df_final["_merge"] != "both"
]
```

Out[169...

	product_id	onsale_web	price	stock_quantity	stock_status	purchase_price	id_web	sku	total_sales	tax_status	post_author	p
81	4741	0	12.4	0	outofstock	6.66	12601	NaN	NaN	NaN	NaN	
127	5957	0	39.0	0	outofstock	20.75	13577	NaN	NaN	NaN	NaN	
139	4289	0	22.8	0	outofstock	11.90	13771	NaN	NaN	NaN	NaN	
180	4869	0	17.2	0	outofstock	9.33	14360	NaN	NaN	NaN	NaN	
185	5955	0	27.3	0	outofstock	13.68	14377	NaN	NaN	NaN	NaN	
...	...	...	...	...	...	...	...	...	...	...	...	
820	7196	0	31.0	55	instock	31.20	NaN	NaN	NaN	NaN	NaN	
821	7200	0	31.0	6	instock	15.54	NaN	NaN	NaN	NaN	NaN	
822	7201	0	31.0	18	instock	16.02	NaN	NaN	NaN	NaN	NaN	
823	7203	0	45.0	30	instock	23.48	NaN	NaN	NaN	NaN	NaN	
824	7204	0	45.0	9	instock	24.18	NaN	NaN	NaN	NaN	NaN	

111 rows × 29 columns



In [170...

```
#Avons-nous des lignes sans correspondance?
if df_final["sku"].isna().any() == True : print("Oui, il y a ",df_final["sku"].isna().sum()," product_id sans correspondance")
else : print("Non, tous les product_id ont bien un sku")
```

Oui, il y a 111 product_id sans correspondance (sans sku)

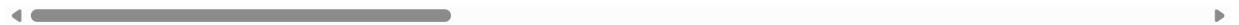
In [171...

```
df_final[df_final["sku"].isna()]
```


Out[171...

	product_id	onsale_web	price	stock_quantity	stock_status	purchase_price	id_web	sku	total_sales	tax_status	post_author	p
81	4741	0	12.4	0	outofstock	6.66	12601	NaN	NaN	NaN	NaN	
127	5957	0	39.0	0	outofstock	20.75	13577	NaN	NaN	NaN	NaN	
139	4289	0	22.8	0	outofstock	11.90	13771	NaN	NaN	NaN	NaN	
180	4869	0	17.2	0	outofstock	9.33	14360	NaN	NaN	NaN	NaN	
185	5955	0	27.3	0	outofstock	13.68	14377	NaN	NaN	NaN	NaN	
...	...	...	...	...	...	...	...	...	...	...	...	
820	7196	0	31.0	55	instock	31.20	NaN	NaN	NaN	NaN	NaN	
821	7200	0	31.0	6	instock	15.54	NaN	NaN	NaN	NaN	NaN	
822	7201	0	31.0	18	instock	16.02	NaN	NaN	NaN	NaN	NaN	
823	7203	0	45.0	30	instock	23.48	NaN	NaN	NaN	NaN	NaN	
824	7204	0	45.0	9	instock	24.18	NaN	NaN	NaN	NaN	NaN	

111 rows × 29 columns



In [172...

```
# Thomas : on remarque que 20 produits présents dans La liste fusionnés ont bien un product_id avec un id_web mais cet id_
# exemple La ligne 823 avec La reference produit '7329', qui a bien un sku/id_web mais ce dernier n'est pas présent.
df_final.loc[
    (df_final["sku"].isna()) &
    (df_final["id_web"].notna()),
    ["product_id", "id_web"]
]
```

Out[172...

	product_id	id_web
81	4741	12601
127	5957	13577
139	4289	13771
180	4869	14360
185	5955	14377
186	5953	14379
212	5505	14648
218	5800	14689
224	5559	14715
227	5570	14730
236	4584	14785
280	4568	15065
303	4864	15154
332	5018	15272
412	6100	15529
428	4922	15586
430	4921	15608
431	5954	15609
438	5021	15630
732	7329	14680-1

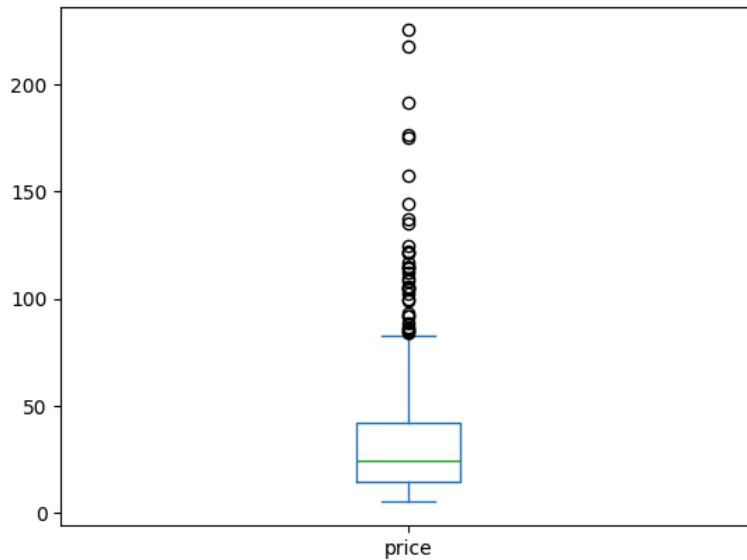
Etape 4 - Analyse univariée des prix

Etape 4.1 - Exploration par la visualisation de données

In [173...

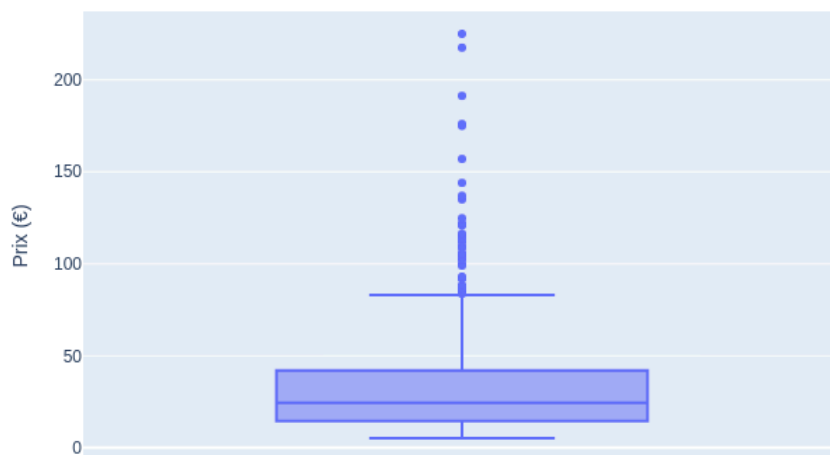
```
#Création d'une boîte à moustache de La répartition des prix grâce à Pandas
df_final.loc[df_final["price"] > 0, "price"].plot.box()
```

Out[173... <Axes: >



```
In [174... #Autre méthode avec plotly express
import plotly.express as px
px.box(
    df_final[df_final["price"] > 0],
    y="price",
    title="Répartition des prix (boxplot)",
    labels={"price": "Prix (€)"}
)
```

Répartition des prix (boxplot)



Etape 4.2 - Exploration par l'utilisation de méthodes statistiques

Etape 4.2.1 - Identification par le Z-index

```
In [175... #Calculer la moyenne du prix

moyenne = df_final.loc[
    (df_final["price"]>0)&
    (df_final["price"].notna()),
    "price"
].mean()

print("Moyenne : ", round(moyenne,2))
```

Moyenne : 32.35

In [176... *# Thomas : Aperçu des prix erronés*

```
liste_prix_erronés = df_final[df_final["price"]<=0]
liste_prix_erronés
```

Out[176...

	product_id	onsale_web	price	stock_quantity	stock_status	purchase_price	id_web	sku	total_sales	tax_status	post_author	p
739	4233	0	-20.0	0	outofstock	10.33	NaN	NaN	NaN	NaN	NaN	
760	5017	0	-8.0	0	outofstock	4.34	NaN	NaN	NaN	NaN	NaN	
772	6594	0	-9.1	19	instock	4.61	NaN	NaN	NaN	NaN	NaN	



In [177... *#Calculer L'écart-type du prix*

```
Ecart_type = df_final.loc[
    (df_final["price"]>0)&(df_final["price"].notna()),"price"].std()

print("Ecart-type : ",round(Ecart_type,2))
```

Ecart-type : 26.62

In [178... *#Calculer Le Z-score*
Thomas : Le Z-score mesure à quelle distance de la moyenne se trouve chaque prix, en nombre d'écarts-types.
(Prix - moyenne) / Ecart-Type

```
# Puis je calcule le z-score
df_final["z_score"] = round(((df_final["price"] - moyenne) / Ecart_type),2)
df_final.head()
```

Pour simplifier j'exclue définitivement les 3 lignes qui peuvent biaiser le calcul (prix erronés ou prix vide), avant d

Out[178...

	product_id	onsale_web	price	stock_quantity	stock_status	purchase_price	id_web	sku	total_sales	tax_status	post_author	post
0	4729	1	8.6	26	instock	4.22	38	38	10.0	taxable	2.0	20112
1	4634	1	41.0	11	instock	20.12	41	41	6.0	taxable	2.0	20112
2	4141	1	39.0	123	instock	24.86	304	304	8.0	taxable	2.0	20112
3	5932	1	59.9	13	instock	27.18	523	523	0.0	taxable	2.0	20115

product_id	onsale_web	price	stock_quantity	stock_status	purchase_price	id_web	sku	total_sales	tax_status	post_author	post

4	5047	1	22.5	76	instock	13.78	531	531	8.0	taxable	2.0	201
												15

```
In [179... #Quel est le seuil prix dont le z-score est supérieur à 3?

#Thomas : Le seuil de prix correspondant à un Z-score supérieur à 3
# est égal à La moyenne augmentée de trois fois L'écart-type.
# Les produits dont le prix dépasse ce seuil peuvent être considérés comme des valeurs "éloignées de la moyenne".

seuil_prix = moyenne + 3 * Ecart_type
print("prix seuil : ",round(seuil_prix,2))
```

prix seuil : 112.22

```
In [180... # Affiche des articles avec le Z-score :

df_final.loc[
    df_final["price"] > seuil_prix,
    ["product_id", "post_title", "price", "purchase_price", "z_score"]
]
```

	product_id	post_title	price	purchase_price	z_score
24	4402	Cognac Frapin VIP XO	176.0	78.25	5.40
33	4406	Cognac Frapin Château de Fontpinot 1989 20 Ans d'Age	157.0	69.08	4.68
87	6215	Domaine des Comtes Lafon Volnay 1er Cru Champans 2014	115.0	56.45	3.10
157	6212	Domaine des Comtes Lafon Volnay 1er Cru Santenots du Milieu 2015	115.0	59.42	3.10
171	4904	Domaine Des Croix Corton Charlemagne Grand Cru 2016	137.0	67.95	3.93
204	5001	David Duband Charmes-Chambertin Grand Cru 2014	217.5	116.87	6.95
233	5918	Wemyss Malts Single Cask Scotch Whisky Chai Caramel Latte 2002 Craigellachie	114.0	52.25	3.07
235	5917	Wemyss Malts Single Cask Scotch Whisky Choc 'n' Nut Pretzel 2001 Bunnahabhain	122.0	54.24	3.37
256	5612	Domaine Weinbach Gewurztraminer Grand Cru Furstentum SGN 2010 1/2	124.8	66.41	3.47
257	6126	Champagne Gosset Célébris Vintage 2007	135.0	80.33	3.86
269	5892	Coteaux Champenois Egly-Ouriet Ambonnay Rouge 2016	191.3	116.06	5.97
281	6216	Domaine des Comtes Lafon Volnay 1er Cru Champans 2016	121.0	60.02	3.33
282	6213	Domaine des Comtes Lafon Volnay 1er Cru Santenots du Milieu 2016	121.0	63.14	3.33
291	6202	Domaine Clerget Echezeaux Grand Cru En Orveaux 2015	116.4	63.15	3.16
313	5767	Camille Giroud Clos de Vougeot 2016	175.0	90.42	5.36
567	4352	Champagne Egly-Ouriet Grand Cru Millésimé 2008	225.0	137.81	7.24
745	4594	NaN	144.0	87.36	4.19

Etape 4.2.2 - Identification par l'intervalle interquartile

```
In [181... #Utilisation de La fonction "describe" de Pandas pour L'étude des mesures de dispersion
df_final_describe = df_final["price"].describe()
df_final_describe
```

```
Out[181...
           price
count  825.000000
mean    32.187697
std     26.712077
min    -20.000000
25%     14.500000
50%     24.300000
75%     42.000000
max    225.000000
```

dtype: float64

```
In [182... #Définir un seuil pour Les articles "outliers" en prix

seuil_outliers = df_final_describe.loc['75%'] + (df_final_describe.loc['75%'] - df_final_describe.loc['25%']) * 1.5
print("Seuil : ",seuil_outliers)
```

Seuil : 83.25

```
In [183... #Définir Le nombre d'articles et La proportion de L'ensemble du catalogue "outliers"

nb_total_articles = len(df_final)
print("Nombre total d'articles : ",nb_total_articles)

nb_articles_outliers = len(df_final[df_final["price"] > seuil_outliers])
print("Nombre d'articles en outliers : ",nb_articles_outliers)

proportion_outliers = nb_articles_outliers / nb_total_articles
print("Proportion d'articles en outliers : ",round(proportion_outliers * 100,2),"%")
```

Nombre total d'articles : 825
Nombre d'articles en outliers : 36
Proportion d'articles en outliers : 4.36 %

```
In [184... #Selon vous, ces outliers sont-ils justifiés ? Comment Le démontrer si cela est possible ?
df_final.loc[df_final["price"] > seuil_outliers, ["product_id","post_title","total_sales","onsale_web","stock_quantity","p

#Ces outliers correspondent aux articles dont Les prix sont Les plus élevés.
#Ils se situent donc naturellement dans La liste des articles qui ont un z-score à supérieur à 3.
#Au regard de la marge commerciale appliquée, celle-ci est identique aux autres articles qui sont en vente.
# Voir si pour ces articles La même marge commerciale doit être conservée (env 50%), La marge en % est à mon sens trop éle
```

Out[184...

	product_id	post_title	total_sales	onsale_web	stock_quantity	purchase_price	price	z_score
22	4404	Cognac Frapin Château de Fontpinot XO	4.0	1	17	52.22	108.5	2.86
23	4407	Cognac Frapin Cigar Blend	5.0	1	14	46.71	104.0	2.69
24	4402	Cognac Frapin VIP XO	3.0	1	11	78.25	176.0	5.40
33	4406	Cognac Frapin Château de Fontpinot 1989 20 Ans d'Age	4.0	1	12	69.08	157.0	4.68
53	6214	Domaine des Comtes Lafon Volnay 1er Cru Champans 2013	6.0	1	9	49.62	99.0	2.50
54	5008	Domaine des Comtes Lafon Volnay 1er Cru Santenots du Milieu 2013	7.0	1	12	56.42	105.0	2.73
56	4132	Zind-Humbrecht Pinot Gris Grand Cru Rangen De Thann Clos Saint-Urbain 2012	5.0	1	7	44.30	88.4	2.11
87	6215	Domaine des Comtes Lafon Volnay 1er Cru Champans 2014	4.0	1	14	56.45	115.0	3.10
88	5007	Domaine des Comtes Lafon Volnay 1er Cru Santenots du Milieu 2014	3.0	1	15	55.88	105.0	2.73
89	4582	Château de Meursault Puligny-Montrachet 1er Cru Champ Canet 2014	1.0	1	18	53.80	109.6	2.90
143	4359	Champagne Larmandier-Bernier Grand Cru Les Chemins d'Avize 2011	7.0	1	112	51.93	85.6	2.00
149	5026	Champagne Agrapart & Fils Minéral Extra Brut Blanc de Blancs Grand Cru 2012	9.0	1	101	50.13	86.8	2.05
150	5025	Champagne Agrapart & Fils L'Avizoise Extra Brut Blanc de Blancs Grand Cru 2012	6.0	1	136	68.60	112.0	2.99
156	5580	Lucien Boillot Puligny-Montrachet 1er Cru Les Perrières 2016	2.0	1	4	41.08	83.7	1.93
157	6212	Domaine des Comtes Lafon Volnay 1er Cru Santenots du Milieu 2015	7.0	1	16	59.42	115.0	3.10
171	4904	Domaine Des Croix Corton Charlemagne Grand Cru 2016	3.0	1	9	67.95	137.0	3.93
204	5001	David Duband Charmes-Chambertin Grand Cru 2014	2.0	1	18	116.87	217.5	6.95
205	6201	David Duband Chambolle-Musigny 1er Cru Les Sentiers 2016	6.0	1	16	57.29	105.6	2.75
233	5918	Wemyss Malts Single Cask Scotch Whisky Chai Caramel Latte 2002 Craigellachie	3.0	1	12	52.25	114.0	3.07
234	5916	Wemyss Malts Single Cask Scotch Whisky Chocolate Moka Cake 2005 Strathclyde	1.0	1	1	40.49	93.0	2.28
235	5917	Wemyss Malts Single Cask Scotch Whisky Choc 'n' Nut Pretzel 2001 Bunnahabhain	3.0	1	12	54.24	122.0	3.37
240	4903	Domaine Des Croix Corton Grand Cru Les Grèves 2017	2.0	1	12	51.80	102.3	2.63
256	5612	Domaine Weinbach Gewurztraminer Grand Cru Furstentum SGN 2010 1/2	1.0	1	19	66.41	124.8	3.47
257	6126	Champagne Gosset Célébris Vintage 2007	5.0	1	138	80.33	135.0	3.86
269	5892	Coteaux Champenois Egly-Ouriet Ambonnay Rouge 2016	6.0	1	98	116.06	191.3	5.97
281	6216	Domaine des Comtes Lafon Volnay 1er Cru Champans 2016	2.0	1	14	60.02	121.0	3.33
282	6213	Domaine des Comtes Lafon Volnay 1er Cru Santenots du Milieu 2016	3.0	1	9	63.14	121.0	3.33
291	6202	Domaine Clerget Echezeaux Grand Cru En Orveaux 2015	5.0	1	12	63.15	116.4	3.16
313	5767	Camille Giroud Clos de Vougeot 2016	4.0	1	12	90.42	175.0	5.36
370	4115	Zind-Humbrecht Riesling Grand Cru Rangen De Thann Clos Saint-Urbain 2017	1.0	1	12	52.70	100.0	2.54
567	4352	Champagne Egly-Ouriet Grand Cru Millésimé 2008	11.0	1	0	137.81	225.0	7.24
730	5565	Tempier Bandol Cabassaou 2017	1.0	1	0	46.11	92.0	2.24
734	4055	NaN	NaN	0	0	37.88	86.1	2.02

	product_id	post_title	total_sales	onsale_web	stock_quantity	purchase_price	price	z_score
745	4594	NaN	NaN	1	0	87.36	144.0	4.19
762	5070	NaN	NaN	1	0	47.43	84.7	1.97
770	6324	NaN	NaN	0	18	99.00	92.0	2.24

Etape 5 - Analyse univariée du CA, des quantités vendues, des stocks et de la marge ainsi qu'une analyse multivariée

Etape 5.1 - Analyse des ventes en CA

```
In [185... #####
# Calculer Le CA du site web #
#####

#Créer une colonne calculant Le CA par article
df_final["ca_article"] = df_final["price"] * df_final["total_sales"]
df_final[["product_id", "post_title", "total_sales", "price", "ca_article"]].head()

#Calculer La somme de la colonne "ca_par_article"
print(df_final["ca_article"].sum(), " €")

#Ce résultat correspond au chiffre d'affaire du site web
```

143680.1 €

```
In [186... #####
# Palmarès des articles en CA #
#####

#Effectuer Le tri dans L'ordre décroissant du CA du dataset df_merge
df_final = df_final.sort_values(by="ca_article", ascending=False)

#Réinitialiser L'index du dataset par un reset_index
df_final = df_final.reset_index(drop=True)

#Afficher Les 20 premiers articles en CA
df_final[["product_id", "post_title", "total_sales", "ca_article"]].head(20)
```


Out[186...

	product_id	post_title	total_sales	ca_article
0	4352	Champagne Egly-Ouriet Grand Cru Millésimé 2008	11.0	2475.0
1	5892	Coteaux Champenois Egly-Ouriet Ambonnay Rouge 2016	6.0	1147.8
2	4353	Champagne Egly-Ouriet Grand Cru Brut Rosé	14.0	1113.0
3	5826	Agnès Levét Côte Rôtie Améthyste 2017	20.0	824.0
4	6212	Domaine des Comtes Lafon Volnay 1er Cru Santenots du Milieu 2015	7.0	805.0
5	5026	Champagne Agrapart & Fils Minéral Extra Brut Blanc de Blancs Grand Cru 2012	9.0	781.2
6	5008	Domaine des Comtes Lafon Volnay 1er Cru Santenots du Milieu 2013	7.0	735.0
7	5767	Camille Giroud Clos de Vougeot 2016	4.0	700.0
8	6126	Champagne Gosset Célébris Vintage 2007	5.0	675.0
9	5025	Champagne Agrapart & Fils L'Avizoise Extra Brut Blanc de Blancs Grand Cru 2012	6.0	672.0
10	6201	David Duband Chambolle-Musigny 1er Cru Les Sentiers 2016	6.0	633.6
11	4406	Cognac Frapin Château de Fontpinot 1989 20 Ans d'Age	4.0	628.0
12	4647	Bernard Baudry Chinon Rouge La Croix Boissée 2017	22.0	627.0
13	4358	Champagne Larmandier-Bernier Grand Cru Vieilles Vignes du Levant 2008	8.0	616.0
14	4359	Champagne Larmandier-Bernier Grand Cru Les Chemins d'Avize 2011	7.0	599.2
15	6214	Domaine des Comtes Lafon Volnay 1er Cru Champans 2013	6.0	594.0
16	6202	Domaine Clerget Echezeaux Grand Cru En Orveaux 2015	5.0	582.0
17	4350	Champagne Egly-Ouriet Grand Cru Extra Brut V.P.	7.0	556.5
18	4573	Château de Meursault Beaune-Grèves Les Trois Journaux 1er Cru 2015	8.0	537.6
19	4402	Cognac Frapin VIP XO	3.0	528.0

In [187...

```
#Graphique en barre des 20 premiers articles avec plotly express

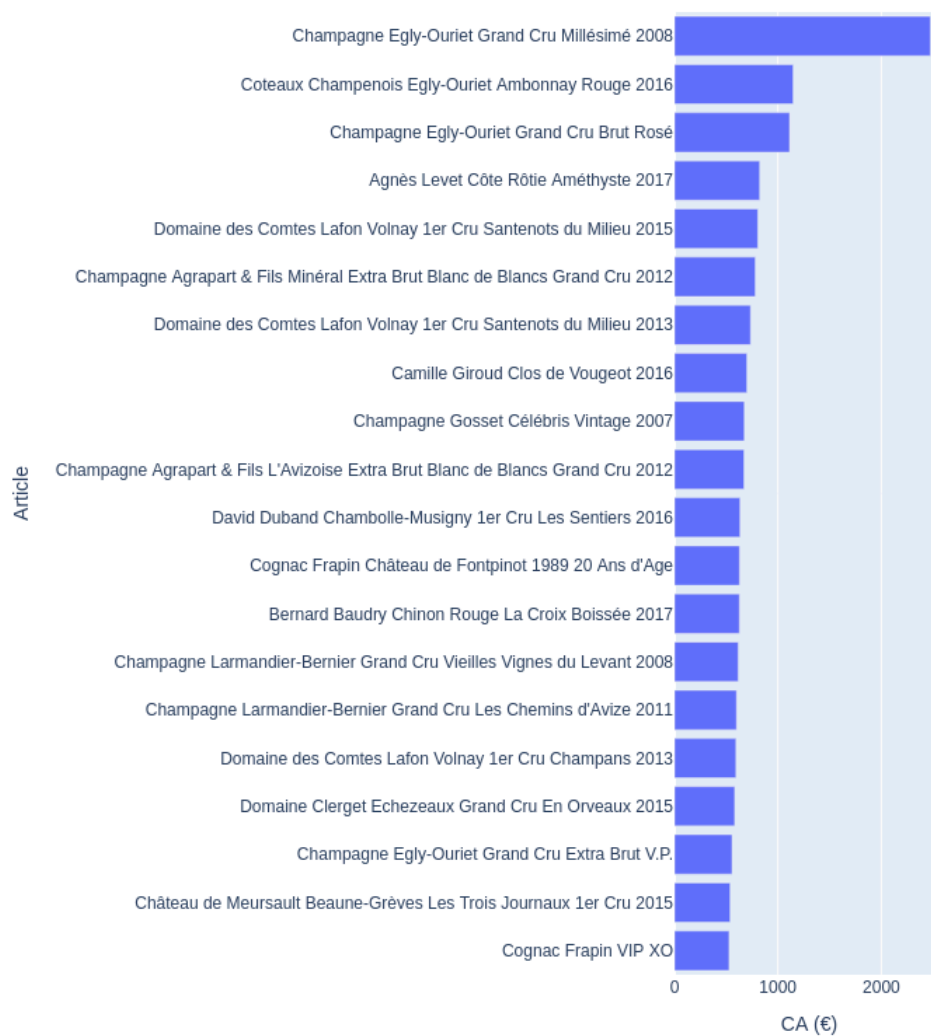
top_20 = df_final.nlargest(20, "ca_article")[["product_id", "post_title", "ca_article"]]

fig = px.bar(
    top_20.sort_values("ca_article"),
    x="ca_article",
    y="post_title",
    orientation="h",
    title="Top 20 des articles par chiffre d'affaires",
    height=800
)

fig.update_layout(
    yaxis_title="Article",
    xaxis_title="CA (€)",
    bargap=0.18,
    margin=dict(l=250, r=30, t=60, b=30) # plus de place pour les libellés à gauche
)

fig.show()
```

Top 20 des articles par chiffre d'affaires

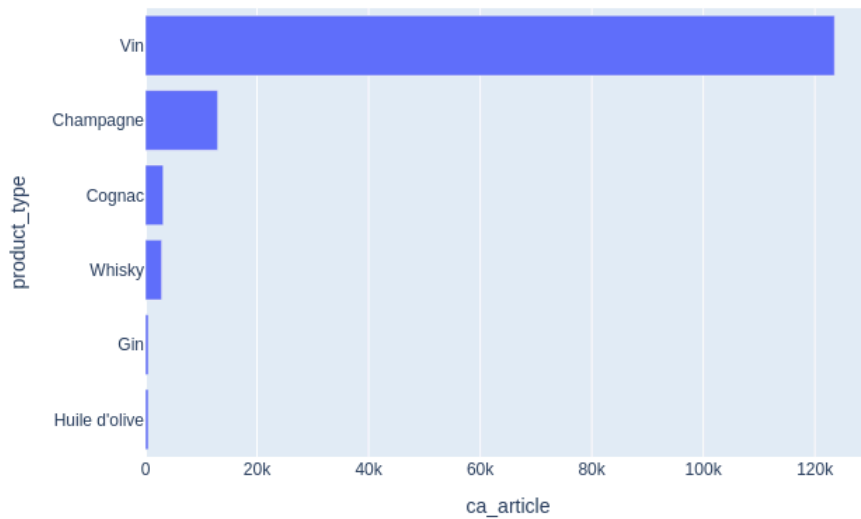


```
In [188... #Ajout Thomas pour présentation - CA par catégorie
ca_par_categorie = (
    df_final
    .groupby("product_type")["ca_article"]
    .sum()
    .reset_index()
    .sort_values("ca_article", ascending=True)
)

fig = px.bar(
    ca_par_categorie,
    x="ca_article",
    y="product_type",
    orientation="h",
    title="Chiffre d'affaires par catégorie"
)

fig.show()
```

Chiffre d'affaires par catégorie

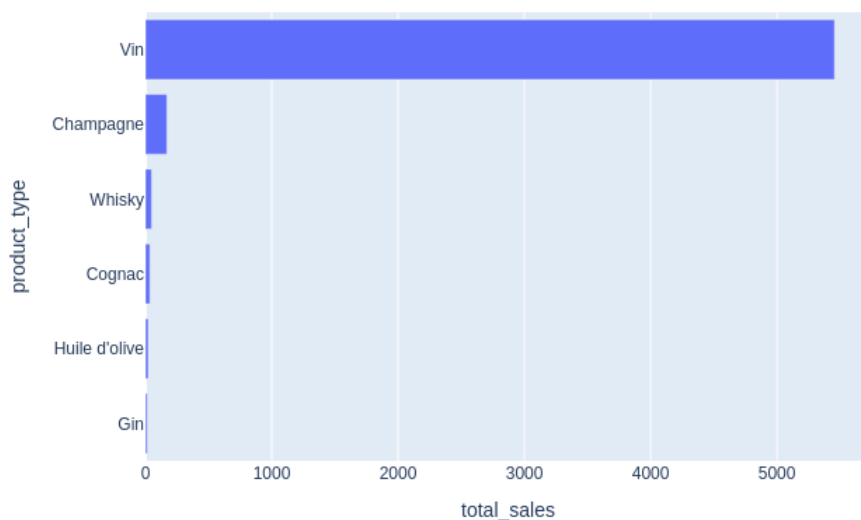


```
In [189... #Ajout Thomas : Quantités vendues par catégorie
qte_par_categorie = (
    df_final
    .groupby("product_type")["total_sales"]
    .sum()
    .reset_index()
    .sort_values("total_sales",ascending=True)
)

fig = px.bar(
    qte_par_categorie,
    x="total_sales",
    y="product_type",
    orientation="h",
    title="Quantités vendues par catégorie"
)

fig.show()
```

Quantités vendues par catégorie



```
In [190... #####
# Calculer le 20 / 80 en CA #
#####

#Créer une colonne calculant la part du CA de la ligne dans le dataset
```

```
ca_total = df_final["ca_article"].sum()
df_final["part_ca"] = df_final["ca_article"] / ca_total

df_final.loc[
    :,
    ["product_id", "post_title", "ca_article", "part_ca"]].head(20)
```

Out[190]...

	product_id	post_title	ca_article	part_ca
0	4352	Champagne Egly-Ouriet Grand Cru Millésimé 2008	2475.0	0.017226
1	5892	Coteaux Champenois Egly-Ouriet Ambonnay Rouge 2016	1147.8	0.007989
2	4353	Champagne Egly-Ouriet Grand Cru Brut Rosé	1113.0	0.007746
3	5826	Agnès Levet Côte Rôtie Améthyste 2017	824.0	0.005735
4	6212	Domaine des Comtes Lafon Volnay 1er Cru Santenots du Milieu 2015	805.0	0.005603
5	5026	Champagne Agrapart & Fils Minéral Extra Brut Blanc de Blancs Grand Cru 2012	781.2	0.005437
6	5008	Domaine des Comtes Lafon Volnay 1er Cru Santenots du Milieu 2013	735.0	0.005116
7	5767	Camille Giroud Clos de Vougeot 2016	700.0	0.004872
8	6126	Champagne Gosset Célébris Vintage 2007	675.0	0.004698
9	5025	Champagne Agrapart & Fils L'Avizoise Extra Brut Blanc de Blancs Grand Cru 2012	672.0	0.004677
10	6201	David Duband Chambolle-Musigny 1er Cru Les Sentiers 2016	633.6	0.004410
11	4406	Cognac Frapin Château de Fontpinot 1989 20 Ans d'Age	628.0	0.004371
12	4647	Bernard Baudry Chinon Rouge La Croix Boissée 2017	627.0	0.004364
13	4358	Champagne Larmandier-Bernier Grand Cru Vieilles Vignes du Levant 2008	616.0	0.004287
14	4359	Champagne Larmandier-Bernier Grand Cru Les Chemins d'Avize 2011	599.2	0.004170
15	6214	Domaine des Comtes Lafon Volnay 1er Cru Champans 2013	594.0	0.004134
16	6202	Domaine Clerget Echezeaux Grand Cru En Orveaux 2015	582.0	0.004051
17	4350	Champagne Egly-Ouriet Grand Cru Extra Brut V.P.	556.5	0.003873
18	4573	Château de Meursault Beaune-Grèves Les Trois Joursaux 1er Cru 2015	537.6	0.003742
19	4402	Cognac Frapin VIP XO	528.0	0.003675

In [191]...

```
#Créer une colonne réalisant la somme cumulative de la colonne précédemment créée

df_final["part_ca_cumulee"] = df_final["part_ca"].cumsum()

df_final.loc[
    :,
    ["post_title", "ca_article", "part_ca", "part_ca_cumulee"]].head(20)
```

Out[191...

	post_title	ca_article	part_ca	part_ca_cumulee
0	Champagne Egly-Ouriet Grand Cru Millésimé 2008	2475.0	0.017226	0.017226
1	Coteaux Champenois Egly-Ouriet Ambonnay Rouge 2016	1147.8	0.007989	0.025214
2	Champagne Egly-Ouriet Grand Cru Brut Rosé	1113.0	0.007746	0.032961
3	Agnès Levet Côte Rôtie Améthyste 2017	824.0	0.005735	0.038696
4	Domaine des Comtes Lafon Volnay 1er Cru Santenots du Milieu 2015	805.0	0.005603	0.044298
5	Champagne Agrapart & Fils Minéral Extra Brut Blanc de Blancs Grand Cru 2012	781.2	0.005437	0.049735
6	Domaine des Comtes Lafon Volnay 1er Cru Santenots du Milieu 2013	735.0	0.005116	0.054851
7	Camille Giroud Clos de Vougeot 2016	700.0	0.004872	0.059723
8	Champagne Gosset Célébris Vintage 2007	675.0	0.004698	0.064421
9	Champagne Agrapart & Fils L'Avizoise Extra Brut Blanc de Blancs Grand Cru 2012	672.0	0.004677	0.069098
10	David Duband Chambolle-Musigny 1er Cru Les Sentiers 2016	633.6	0.004410	0.073508
11	Cognac Frapin Château de Fontpinot 1989 20 Ans d'Age	628.0	0.004371	0.077879
12	Bernard Baudry Chinon Rouge La Croix Boissée 2017	627.0	0.004364	0.082242
13	Champagne Larmandier-Bernier Grand Cru Vieilles Vignes du Levant 2008	616.0	0.004287	0.086530
14	Champagne Larmandier-Bernier Grand Cru Les Chemins d'Avize 2011	599.2	0.004170	0.090700
15	Domaine des Comtes Lafon Volnay 1er Cru Champans 2013	594.0	0.004134	0.094834
16	Domaine Clerget Echezeaux Grand Cru En Orveaux 2015	582.0	0.004051	0.098885
17	Champagne Egly-Ouriet Grand Cru Extra Brut V.P.	556.5	0.003873	0.102758
18	Château de Meursault Beaune-Grèves Les Trois Journaux 1er Cru 2015	537.6	0.003742	0.106500
19	Cognac Frapin VIP XO	528.0	0.003675	0.110175

In [192...

```
#Grâce aux deux colonnes créées précédemment, calculer Le nombre d'articles représentant 80% du CA

nb_articles_total = df_final.shape[0] # → Nb d'article au total
print("Nombre d'articles total : ",nb_articles_total)

#Afficher la proportion que représente ce groupe d'articles dans Le catalogue entier du site web

articles_80 = df_final.loc[df_final["part_ca_cumulee"] < 0.8] # → J'isole Les 80%
nb_articles_80 = articles_80.shape[0]

proportion_articles_80 = nb_articles_80 / nb_articles_total # → Je calcule La proportion
proportion_articles_80_pct = round(proportion_articles_80 * 100, 2) # → Je transforme en pourcentage

print(nb_articles_80," articles permettent d'atteindre 80 % du chiffre d'affaires, ce qui représente",proportion_articles_
```

Nombre d'articles total : 825

434 articles permettent d'atteindre 80 % du chiffre d'affaires, ce qui représente 52.61 % de l'ensemble du catalogue du site web.

Etape 5.2 - Analyse des ventes en quantité

In [193...

```
#####
# Palmarès des articles en quantité #
#####

#Effectuer Le tri dans L'ordre décroissant de quantités vendues du dataset df_merge

df_final_par_qte_vendue = df_final.sort_values(by="total_sales", ascending=False)
df_final_par_qte_vendue[["product_id", "post_title", "total_sales"]].head(20)

#Réinitialiser L'index du dataset par un reset_index
df_final_par_qte_vendue = df_final_par_qte_vendue.reset_index(drop=True)

#Afficher Les 20 premiers articles en quantité
df_final_par_qte_vendue[["product_id", "post_title", "total_sales"]].head(20)
```

Out[193...

	product_id	post_title	total_sales
0	4867	Château De La Selve IGP Coteaux de l'Ardèche Maguelonne Rosé 2019	36.0
1	4203	Mas Laval IGP Pays d'Hérault Les Pampres Blancs 2018	27.0
2	4275	I Fabbri Chianti Classico Lamole 2017	24.0
3	4726	François Baur Pinot Noir Schlittweg 2017	22.0
4	4647	Bernard Baudry Chinon Rouge La Croix Boissée 2017	22.0
5	6129	Moulin de Gassac IGP Pays d'Hérault Guilhem Blanc 2017	20.0
6	5826	Agnès Levet Côte Rôtie Améthyste 2017	20.0
7	4220	Xavier Frissant Touraine Amboise Chenin Les Pierres 2018	18.0
8	5803	Château Tour Haut-Caussan Médoc 2015	17.0
9	6569	Decelle-Villa Chorey-Lès-Beaune 2016	17.0
10	5778	Maurel Pays d'Oc Merlot 2018	17.0
11	4105	Emile Boeckel Sylvaner Vieilles Vignes 2019	16.0
12	4188	Château de La Liquière Languedoc Blanc Les Amandiers 2019	16.0
13	5777	Maurel Pays d'Oc Cabernet-Sauvignon 2017	16.0
14	4863	Château Ollieux Romanis Corbières Rosé Classique 2019	16.0
15	4870	Triennes IGP Méditerranée Rosé 2019	16.0
16	5695	Philippe Nusswitz IGP Cévennes Rosé O Pale 2019	16.0
17	4059	Mourgues du Grès Costières de Nîmes Galets Rosés 2019	16.0
18	4207	Domaine Montrose Côtes de Thongue Rouge 2018	15.0
19	5958	Domaine Labranche Laffont Madiran Tradition 2016	15.0

In [194...

#Graphique en barre des 20 premiers articles avec plotly express

```

top_20_total_sales = df_final_par_qte_vendue.nlargest(20, "total_sales")[["product_id", "post_title", "total_sales"]]

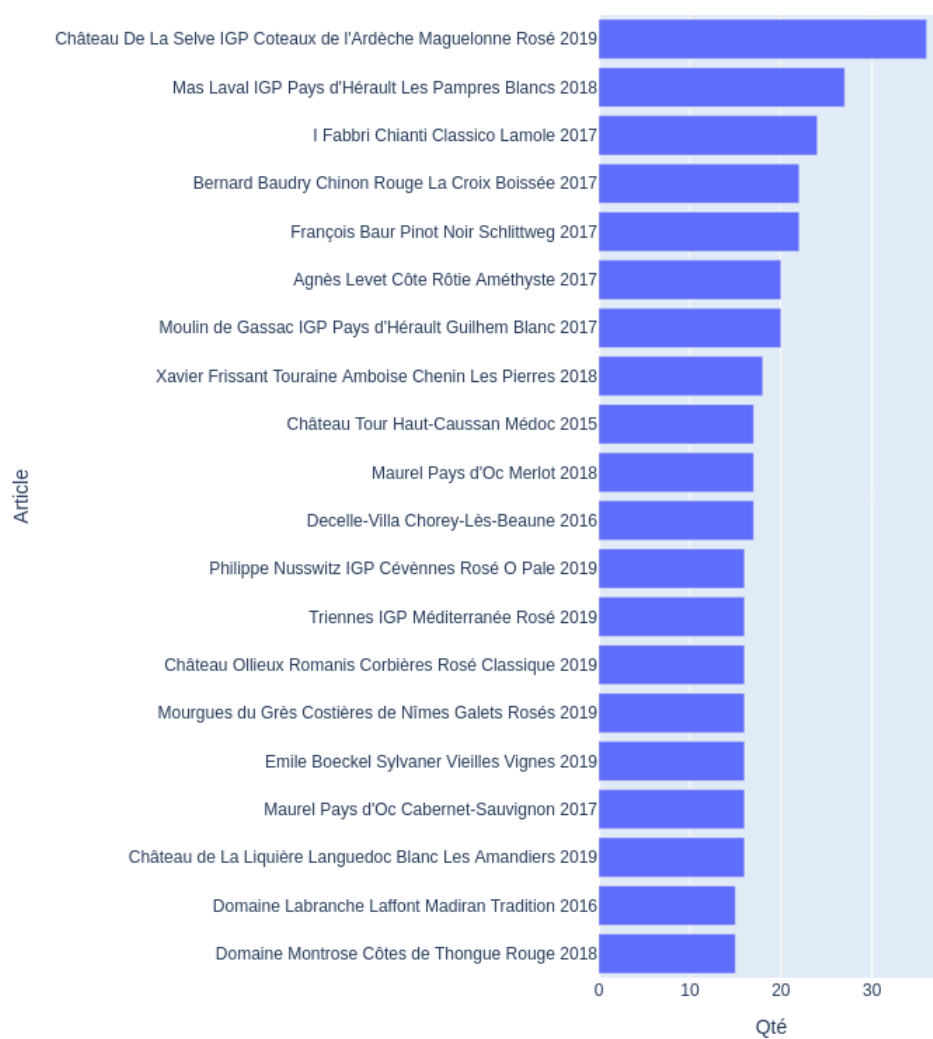
histo_qte = px.bar(
    top_20_total_sales.sort_values("total_sales"),
    x="total_sales",
    y="post_title",
    orientation="h",
    title="Top 20 des articles par quantité vendue",
    height=800
)

histo_qte.update_layout(
    yaxis_title="Article",
    xaxis_title="Qté",
    bargap=0.18,
    margin=dict(l=250, r=30, t=60, b=30) # plus de place pour les libellés à gauche
)

histo_qte.show()

```

Top 20 des articles par quantité vendue



In [195...

```
#####
# Calculer le 20 / 80 en CA #
#####

#Créer une colonne calculant la part en quantité de la ligne dans le dataset

qte_total = df_final_par_qte_vendue["total_sales"].sum()
df_final_par_qte_vendue["Part_qte_totale"] = df_final_par_qte_vendue["total_sales"] / ca_total

df_final_par_qte_vendue.loc[:,["product_id", "post_title", "total_sales", "Part_qte_totale"]].head(20)
```

Out[195...

	product_id	post_title	total_sales	Part_qte_totale
0	4867	Château De La Selve IGP Coteaux de l'Ardèche Maguelonne Rosé 2019	36.0	0.000251
1	4203	Mas Laval IGP Pays d'Hérault Les Pampres Blancs 2018	27.0	0.000188
2	4275	I Fabbri Chianti Classico Lamole 2017	24.0	0.000167
3	4726	François Baur Pinot Noir Schlittweg 2017	22.0	0.000153
4	4647	Bernard Baudry Chinon Rouge La Croix Boissée 2017	22.0	0.000153
5	6129	Moulin de Gassac IGP Pays d'Hérault Guilhem Blanc 2017	20.0	0.000139
6	5826	Agnès Levet Côte Rôtie Améthyste 2017	20.0	0.000139
7	4220	Xavier Frissant Touraine Amboise Chenin Les Pierres 2018	18.0	0.000125
8	5803	Château Tour Haut-Caussan Médoc 2015	17.0	0.000118
9	6569	Decelle-Villa Chorey-Lès-Beaune 2016	17.0	0.000118
10	5778	Maurel Pays d'Oc Merlot 2018	17.0	0.000118
11	4105	Emile Boeckel Sylvaner Vieilles Vignes 2019	16.0	0.000111
12	4188	Château de La Liquière Languedoc Blanc Les Amandiers 2019	16.0	0.000111
13	5777	Maurel Pays d'Oc Cabernet-Sauvignon 2017	16.0	0.000111
14	4863	Château Ollieux Romanis Corbières Rosé Classique 2019	16.0	0.000111
15	4870	Triennes IGP Méditerranée Rosé 2019	16.0	0.000111
16	5695	Philippe Nusswitz IGP Cévennes Rosé O Pale 2019	16.0	0.000111
17	4059	Mourgues du Grès Costières de Nîmes Galets Rosés 2019	16.0	0.000111
18	4207	Domaine Montrose Côtes de Thongue Rouge 2018	15.0	0.000104
19	5958	Domaine Labranche Laffont Madiran Tradition 2016	15.0	0.000104

In [196...

```
#Créer une colonne réalisant la somme cumulative de la colonne précédemment créée

df_final_par_qte_vendue["Part_Cumul_Qte"] = df_final_par_qte_vendue["Part_qte_totale"].cumsum()
df_final_par_qte_vendue.loc[:,["product_id", "post_title", "Part_qte_totale", "Part_Cumul_Qte"]].head(20)
```

Out[196...

	product_id	post_title	Part_qte_totale	Part_Cumul_Qte
0	4867	Château De La Selve IGP Coteaux de l'Ardèche Maguelonne Rosé 2019	0.000251	0.000251
1	4203	Mas Laval IGP Pays d'Hérault Les Pampres Blancs 2018	0.000188	0.000438
2	4275	I Fabbri Chianti Classico Lamole 2017	0.000167	0.000606
3	4726	François Baur Pinot Noir Schlittweg 2017	0.000153	0.000759
4	4647	Bernard Baudry Chinon Rouge La Croix Boissée 2017	0.000153	0.000912
5	6129	Moulin de Gassac IGP Pays d'Hérault Guilhem Blanc 2017	0.000139	0.001051
6	5826	Agnès Levet Côte Rôtie Améthyste 2017	0.000139	0.001190
7	4220	Xavier Frissant Touraine Amboise Chenin Les Pierres 2018	0.000125	0.001315
8	5803	Château Tour Haut-Caussan Médoc 2015	0.000118	0.001434
9	6569	Decelle-Villa Chorey-Lès-Beaune 2016	0.000118	0.001552
10	5778	Maurel Pays d'Oc Merlot 2018	0.000118	0.001670
11	4105	Emile Boeckel Sylvaner Vieilles Vignes 2019	0.000111	0.001782
12	4188	Château de La Liquière Languedoc Blanc Les Amandiers 2019	0.000111	0.001893
13	5777	Maurel Pays d'Oc Cabernet-Sauvignon 2017	0.000111	0.002004
14	4863	Château Ollieux Romanis Corbières Rosé Classique 2019	0.000111	0.002116
15	4870	Triennes IGP Méditerranée Rosé 2019	0.000111	0.002227
16	5695	Philippe Nusswitz IGP Cévennes Rosé O Pale 2019	0.000111	0.002339
17	4059	Mourgues du Grès Costières de Nîmes Galets Rosés 2019	0.000111	0.002450
18	4207	Domaine Montrose Côtes de Thongue Rouge 2018	0.000104	0.002554
19	5958	Domaine Labranche Laffont Madiran Tradition 2016	0.000104	0.002659

In [197...

```
#Grâce aux deux colonnes créées précédemment, calculer le nombre d'articles représentant 80% du CA

nb_articles_total = df_final_par_qte_vendue.shape[0] # → Nb d'article au total
```



```
#Afficher la proportion que représente ce groupe d'articles dans le catalogue entier du site web

Liste_articles_80 = df_final_par_qte_vendue.loc[df_final_par_qte_vendue["Part_Cumul_Qte"] < 0.8] # → J'isole les 80%
nombre_articles_80 = Liste_articles_80.shape[0]

proportion_articles_80_ = nombre_articles_80 / nb_articles_total # → Je calcule la proportion
Prop_articles_80_pct = round(proportion_articles_80_ * 100, 2) # → Je transforme en pourcentage

print(nombre_articles_80, " articles permettent d'atteindre 80 % du chiffre d'affaires, ce qui représente", Prop_articles_80_pct, "% de l'ensemble du catalogue du site web.")
```

714 articles permettent d'atteindre 80 % du chiffre d'affaires, ce qui représente 86.55 % de l'ensemble du catalogue du site web.

In [198...

```
#Sous forme graphique :

# 1) Tri décroissant par CA
df_pareto = df_final_par_qte_vendue.sort_values("ca_article", ascending=False).copy()

# 2) Part de CA
df_pareto["part_ca"] = df_pareto["ca_article"] / df_pareto["ca_article"].sum()

# 3) Part cumulée
df_pareto["part_ca_cumulee"] = df_pareto["part_ca"].cumsum()

# (option) rang produit pour l'axe X
df_pareto["rang"] = range(1, len(df_pareto) + 1)

import plotly.graph_objects as go

fig = go.Figure()

# Barres : CA par article
fig.add_trace(
    go.Bar(
        x=df_pareto["rang"],
        y=df_pareto["ca_article"],
        name="CA par article"
    )
)

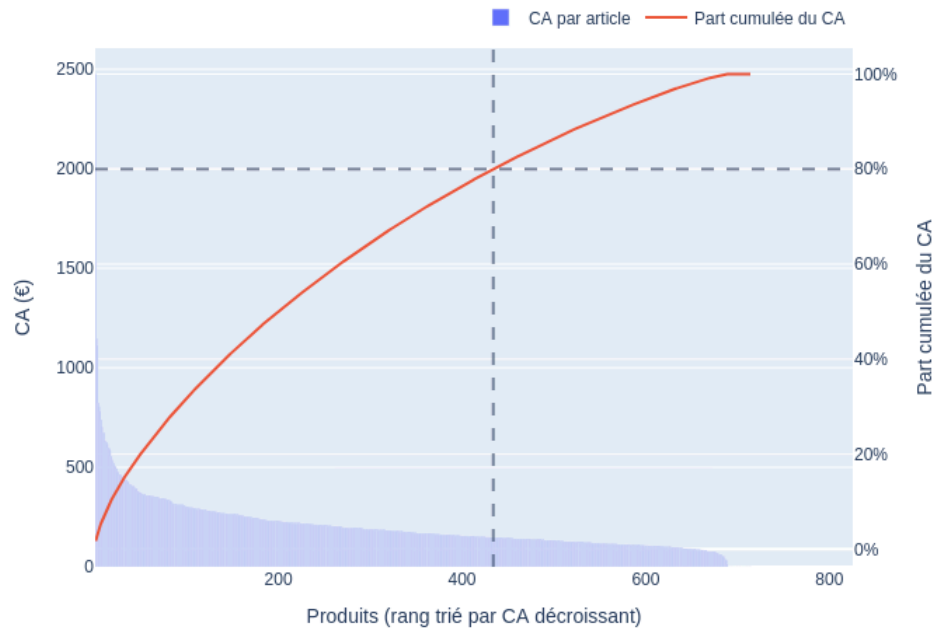
# Courbe : part cumulée du CA
fig.add_trace(
    go.Scatter(
        x=df_pareto["rang"],
        y=df_pareto["part_ca_cumulee"],
        name="Part cumulée du CA",
        mode="lines",
        yaxis="y2"
    )
)

# Ligne horizontale à 80%
fig.add_hline(y=0.8, line_dash="dash", opacity=0.6, yref="y2")

# Ligne verticale au point 80% (si tu veux)
fig.add_vline(x=nb_articles_80, line_dash="dash", opacity=0.6)
fig.update_layout(
    title="Pareto du CA (80/20)",
    xaxis_title="Produits (rang trié par CA décroissant)",
    yaxis_title="CA (€)",
    yaxis2=dict(
        title="Part cumulée du CA",
        overlaying="y",
        side="right",
        tickformat=".0%"
    ),
    legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1),
    height=550
)

fig.show()
```

Pareto du CA (80/20)



Etape 5.3 - Analyse des stocks

```

In [199... #####
# Calculer Le nombre de mois de stock #
#####

#Import de numpy
import numpy as np

#Création de la colonne Mois_en_vente
df_base_finale["post_date"] = pd.to_datetime(df_base_finale["post_date"]) # → Je m'assure que la date est bien au bon form
today = pd.Timestamp.today() # → Je m'assure que la date du jour soit au bon format
df_base_finale["mois_en_vente"] = round(((today - df_base_finale["post_date"]).dt.days / 30),1) # → nb de mois en vente

In [200... moyenne_mois = df_base_finale["mois_en_vente"].mean()
round(moyenne_mois, 1)
print("les articles restent en vente en moyenne ",round(moyenne_mois,1)," en moyenne par article")

#Combien de mois peut-on vendre au rythme actuel avec le stock disponible ?
# 1. Calculer Les ventes mensuelles moyennes

df_base_finale["ventes_mensuelles"] = (df_base_finale["total_sales"] / df_base_finale["mois_en_vente"])

# 2. Calculer Le nombre de mois de stock
df_base_finale["mois_de_stock"] = (df_base_finale["stock_quantity"] / df_base_finale["ventes_mensuelles"])

# 3. Nettoyage des éventuelles mauvaises valeurs
df_base_finale["mois_de_stock"] = (df_base_finale["mois_de_stock"].replace([np.inf, -np.inf], np.nan))

# 4. Calcul de la couverture du stock au global

mois_stock_global = (df_base_finale["stock_quantity"].sum() /
(df_base_finale["total_sales"].sum() / df_base_finale["mois_en_vente"].clip(lower=1, upper=12).mean())
)

print("la couverture de stock est de ",round(mois_stock_global,0)," mois")

les articles restent en vente en moyenne 95.5 en moyenne par article
la couverture de stock est de 37.0 mois

In [201... #Aperçu :
df_base_finale.loc[:,["product_id","post_title","stock_quantity","post_date","total_sales","onsale_web","mois_en_vente","v

```

Out[201...

	product_id	post_title	stock_quantity	post_date	total_sales	onsale_web	mois_en_vente	ventes_mensuelles
0	4729	Emile Boeckel Crémant Brut Blanc de Blancs	26	2018-04-18 12:25:58	10.0	1	99.7	0.100301
1	4634	Marcel Windholtz Eau de Vie de Marc de Gewurztraminer	11	2018-04-14 12:01:43	6.0	1	99.8	0.060120
2	4141	Champagne Gosset Grande Réserve	123	2018-02-13 12:57:44	8.0	1	101.8	0.078585
3	5932	Cognac Normandin Mercier VFC	13	2019-04-06 15:25:58	0.0	1	87.9	0.000000
4	5047	Champagne Petit Lebrun & Fils Blanc de Blancs Grand Cru	76	2018-07-18 15:58:02	8.0	1	96.7	0.082730

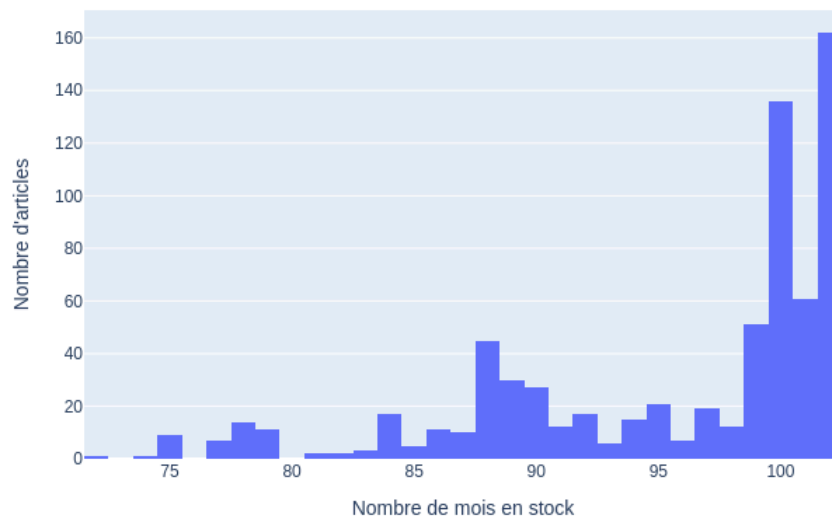
In [202...

```
fig = px.histogram(
    df_base_finale,
    x="mois_en_vente",
    nbins=30,
    title="Distribution des mois en stock"
)

fig.update_layout(
    xaxis_title="Nombre de mois en stock",
    yaxis_title="Nombre d'articles"
)

fig.show()
```

Distribution des mois en stock



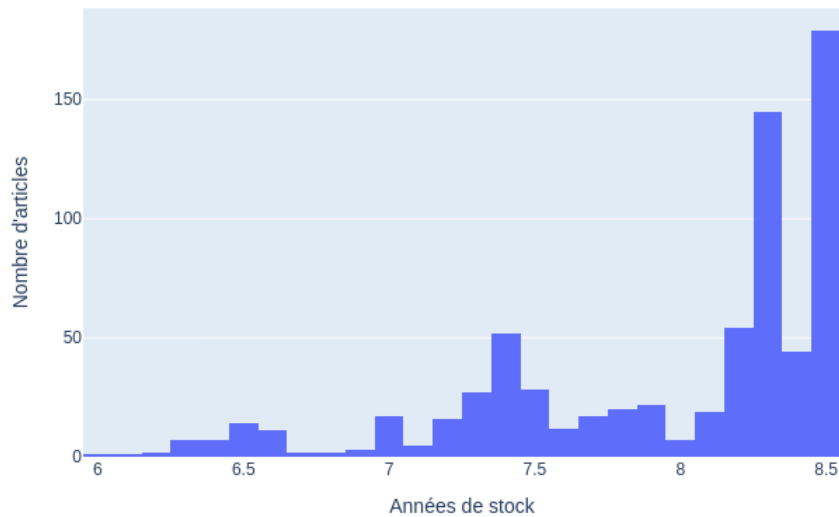
In [203...

```
# conversion en année pour meilleure visu
df_base_finale["annees_stock"] = (df_base_finale["mois_en_vente"] / 12).round(2)
fig = px.histogram(
    df_base_finale,
    x=df_base_finale["mois_en_vente"] / 12,
    nbins=30,
    title="Distribution des années de stock"
)

fig.update_layout(
    xaxis_title="Années de stock",
    yaxis_title="Nombre d'articles"
)

fig.show()
```

Distribution des années de stock



```
In [204...] #Remplacement des "inf" par 0
df_base_finale["mois_en_vente"] = df_base_finale["mois_en_vente"].replace([np.inf, -np.inf], 0)
df_analyse_stock = df_base_finale.copy()
```

```
In [205...] #Effectuer Le tri dans l'ordre décroissant du nombre de mois de stock dans Le dataset df_merge
table_rotations = df_analyse_stock.loc[:,
    ["product_id", "post_title", "stock_quantity", "total_sales", "onsale_web", "mois_en_vente"]
    ].sort_values(by="mois_en_vente", ascending=False)

#Aperçu :
table_rotations = table_rotations.reset_index(drop=True)
table_rotations.head(20)
```

	product_id	post_title	stock_quantity	total_sales	onsale_web	mois_en_vente
0	3850	Pierre Jean Villa Crozes-Hermitage Accroche Coeur 2018	0	0.0	1	102.0
1	3847	Pierre Jean Villa Saint-Joseph Préface 2018	16	6.0	1	102.0
2	3849	Pierre Jean Villa Saint-Joseph Rouge Tildé 2017	10	9.0	1	102.0
3	4032	Pierre Jean Villa IGP Collines Rhodaniennes Gamine 2018	26	12.0	1	102.0
4	4040	Pierre Jean Villa Saint-Joseph Saut De l'Ange 2018	12	7.0	1	101.9
5	4059	Mourgues du Grès Costières de Nîmes Galets Rosés 2019	34	16.0	1	101.9
6	4062	Mourgues du Grès IGP Pont du Gard Terre d'Argence 2018	30	9.0	1	101.9
7	4047	Pierre Gaillard Côtes-du-Rhône Blanc Les Gendrines 2018	0	0.0	1	101.9
8	4043	Pierre Gaillard Côte Rôtie Esprit de Blonde 2017	0	0.0	1	101.9
9	4041	Pierre Gaillard Condrieu 2018	12	6.0	1	101.9
10	4046	Pierre Gaillard Côte Rôtie Rose Pourpre 2017	2	2.0	1	101.9
11	4069	Gilles Robin Hermitage Rouge 2012	4	3.0	1	101.9
12	4072	Château de Vaudieu Châteauneuf-du-Pape Blanc 2015	8	5.0	1	101.9
13	4065	Oratoire Saint Martin Cairanne Rouge Les Douyes 2016	0	0.0	1	101.9
14	4076	Plateau des Chênes Lirac 2016	32	13.0	1	101.9
15	4057	Mourgues du Grès Costières de Nîmes Galets Dorés 2019	29	14.0	1	101.9
16	4052	Clos du Mont-Olivet Châteauneuf-du-Pape Blanc 2019	0	0.0	1	101.9
17	4063	Mourgues du Grès Costières de Nîmes Terre de Feu 2017	33	12.0	1	101.9
18	4051	Jeanne Gaillard IGP Collines Rhodaniennes Syrah Terre de Mandrin 2018	0	0.0	1	101.9
19	4073	Château de Vaudieu Châteauneuf-du-Pape L'Avenue 2015	8	4.0	1	101.9

```

In [206... #Graphique en barre du flop 20 des produits qui ont le plus de mois de stock

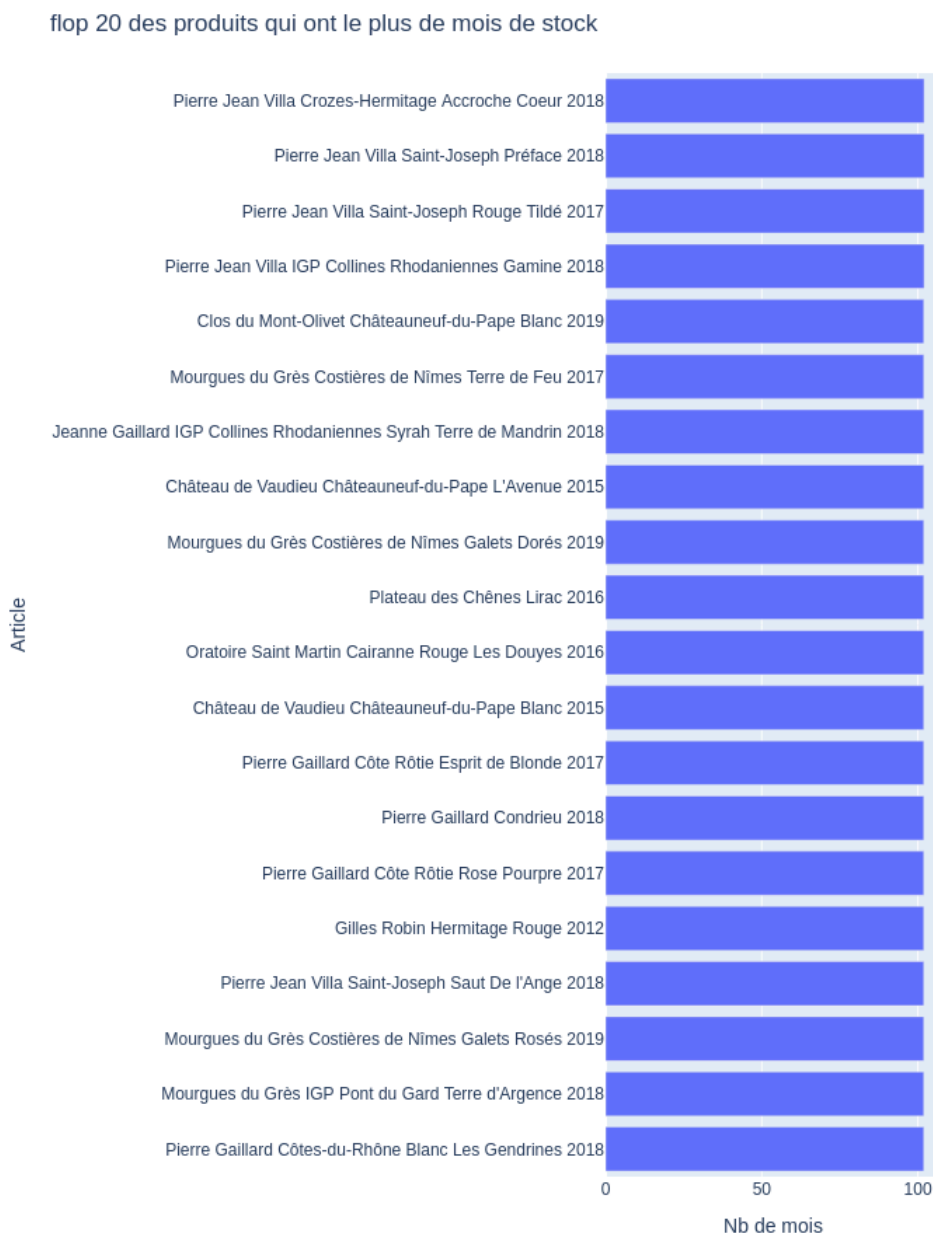
table_rotations = table_rotations.head(20)
table_rotations.nlargest(20, "mois_en_vente")[["product_id", "post_title", "mois_en_vente"]]

histo_qte = px.bar(
    table_rotations.sort_values("mois_en_vente"),
    x="mois_en_vente",
    y="post_title",
    orientation="h",
    title="flop 20 des produits qui ont le plus de mois de stock",
    height=900
)

histo_qte.update_layout(
    yaxis_title="Article",
    xaxis_title="Nb de mois",
    bargap=0.2,
    margin=dict(l=320, r=30, t=60, b=40) # plus de place pour les libellés à gauche
)

histo_qte.show()

```



```

In [207... #####
# Valorisation des stocks en euros #
#####

#Création de la colonne Valorisation des stocks en euros
df_base_finale["Valeur_stock_comptable"] = df_base_finale["purchase_price"] * df_base_finale["stock_quantity"]
df_base_finale["Valeur_stock_commerciale"] = df_base_finale["price"] * df_base_finale["stock_quantity"]
df_base_finale.loc[:,["product_id", "post_title", "stock_quantity", "purchase_price", "Valeur_stock_comptable", "Valeur_stock_c

```

Out [207...

	product_id	post_title	stock_quantity	purchase_price	Valeur_stock_comptable	Valeur_stock_commerciale
0	4729	Emile Boeckel Crémant Brut Blanc de Blancs	26	4.22	109.72	223.6
1	4634	Marcel Windholtz Eau de Vie de Marc de Gewurztraminer	11	20.12	221.32	451.0
2	4141	Champagne Gosset Grande Réserve	123	24.86	3057.78	4797.0
3	5932	Cognac Normandin Mercier VFC	13	27.18	353.34	778.7
4	5047	Champagne Petit Lebrun & Fils Blanc de Blancs Grand Cru	76	13.78	1047.28	1710.0
...	...	...	...	...	...	...
820	7196	NaN	55	31.20	1716.00	1705.0
821	7200	NaN	6	15.54	93.24	186.0
822	7201	NaN	18	16.02	288.36	558.0
823	7203	NaN	30	23.48	704.40	1350.0
824	7204	NaN	9	24.18	217.62	405.0

825 rows × 6 columns

In [208...

```
#Calculer la somme de la colonne "Valorisation_stock_euros"
print(df_base_finale["Valeur_stock_comptable"].sum()," € (vision comptable)")
print(df_base_finale["Valeur_stock_commerciale"].sum()," € (vision commerciale)")
```

```
298555.76 € (vision comptable)
531628.8 € (vision commerciale)
```

In [209...

```
#####
# Valorisation du nombre de produits en stock #
#####

#Calculer la somme de la colonne stock quantity
print(df_base_finale["stock_quantity"].sum()," articles en stock")
```

17811 articles en stock

Etape 5.4 - Analyse du taux de marge

In [210...

```
#####
# Analyse du taux de marge #
#####

#Création de la colonne Prix HT

df_base_finale["prix_HT"] = (df_base_finale["price"] / 1.20).round(2)

#Création de la colonne Taux de marge
df_base_finale["taux_marge"] = (
    (df_base_finale["price"] - df_base_finale["purchase_price"]) / df_base_finale["purchase_price"]
)
```

In [211...

```
#Afficher le prix minimum de la colonne "taux_marge"
taux_marge_min = df_base_finale["taux_marge"].min()
print(taux_marge_min)
#Afficher le prix maximum de la colonne "taux_marge"
taux_marge_max = df_base_finale["taux_marge"].max()
print(taux_marge_max)
```

```
-2.9739696312364425
1.296949650863653
```

In [212...

```
#Affichage de la ligne avec un taux de marge inférieur à 0
df_base_finale.loc[
    df_base_finale["taux_marge"] < 0,
    ["product_id", "post_title", "price", "prix_HT", "purchase_price", "onsale_web", "taux_marge"]
]
```

Out[212...

	product_id		post_title	price	prix_HT	purchase_price	onsale_web	taux_marge
79	4355	Champagne Egly-Ouriet Grand Cru Blanc de Noirs		12.65	10.54	77.48	1	-0.836732
303	4864		NaN	8.30	6.92	9.99	0	-0.169169
739	4233		NaN	-20.00	-16.67	10.33	0	-2.936108
760	5017		NaN	-8.00	-6.67	4.34	0	-2.843318
770	6324		NaN	92.00	76.67	99.00	0	-0.070707
772	6594		NaN	-9.10	-7.58	4.61	0	-2.973970
820	7196		NaN	31.00	25.83	31.20	0	-0.006410

In [213...

```
#Création d'un dataframe avec Les taux positifs
df_marge_positive = df_base_finale.loc[
    df_base_finale["taux_marge"] > 0
].copy()

#Afficher Le prix minimum de la colonne "taux_marge"
taux_marge_min = df_marge_positive["taux_marge"].min()
print(f"Taux de marge minimum (positif) : {taux_marge_min:.2%}")

#Afficher Le prix maximum de la colonne "taux_marge"
taux_marge_max = df_marge_positive["taux_marge"].max()
print(f"Taux de marge maximum : {taux_marge_max:.2%}")
```

Taux de marge minimum (positif) : 55.40%
Taux de marge maximum : 129.69%

In [214...

```
#Création d'un dataframe avec Le taux de marge moyen par type de produit
taux_marge_par_type = df_marge_positive.groupby("product_type")["taux_marge"].mean()
```

In [215...

```
#Affichage dans un graphique du taux de marge par type de produit

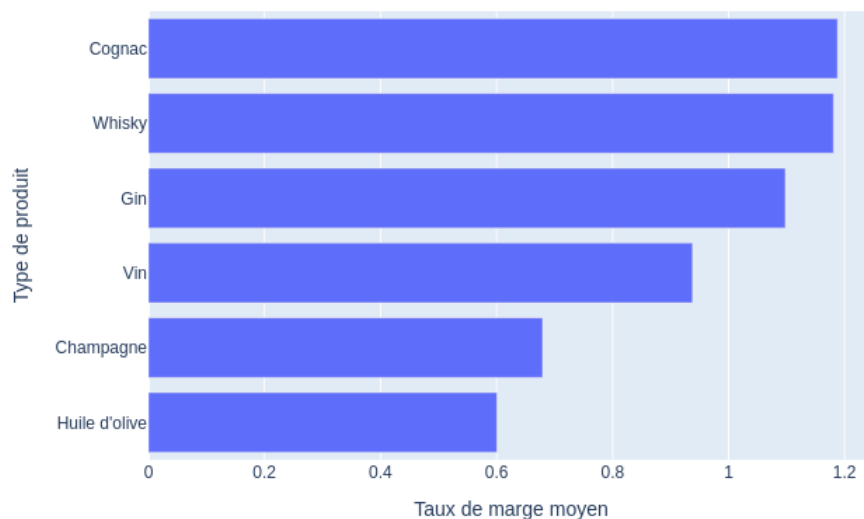
taux_marge_par_type_tri = taux_marge_par_type.sort_values() # trie par valeurs (marges)

fig = px.bar(
    x=taux_marge_par_type_tri.values,
    y=taux_marge_par_type_tri.index,
    orientation="h",
    title="Taux de marge moyen par type de produit",
    height=500
)

fig.update_layout(
    xaxis_title="Taux de marge moyen",
    yaxis_title="Type de produit",
    bargap=0.2
)

fig.show()
```

Taux de marge moyen par type de produit



Etape 5.5 - Analyse des corrélations entre les variables stock, sales et price

```
In [216... #####
# Analyse des corrélations #
#####

#Importation de Seaborn
import seaborn as sns

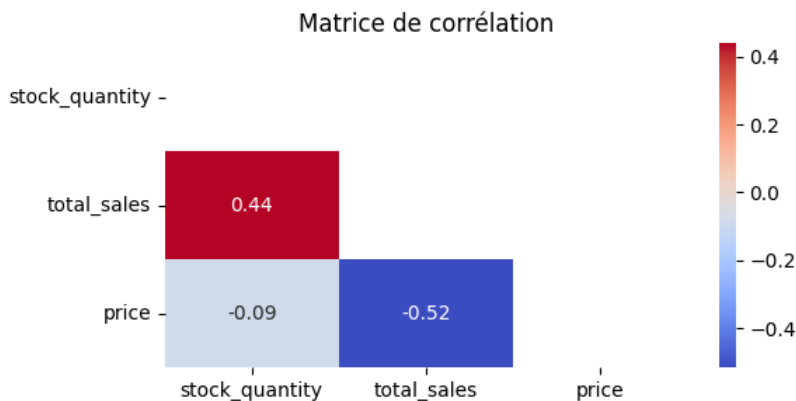
#Création d'une heatmap de corrélation avec les variables stock, sales et price
#On peut également créer un mask pour n'afficher qu'une demi heatmap

import seaborn as sns
import matplotlib.pyplot as plt

corr_matrix = df_base_finale[["stock_quantity", "total_sales", "price"]].corr()

mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

plt.figure(figsize=(6, 3))
sns.heatmap(
    corr_matrix,
    mask=mask,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)
plt.title("Matrice de corrélation")
plt.show()
```



```
In [217... #Que peut-on conclure des corrélations ?

# Corrélation stock_quantity & total_sales - Corrélation positive modérée (+0,44)
# Les produits qui se vendent le plus ont également le plus de stock.
# C'est plutôt logique, Le stock suit la demande.

# Corrélation price & total_sales - Corrélation négative relativement forte (-0,25)
# Plus le prix est élevé, moins les ventes sont importantes
# La société vend d'avantage de vins & spiritueux grand public

# Corrélation price & stock_quantity - Corrélation très faible (-0,09)
# Le niveau de stock n'est pas directement lié au prix
# Le stock est davantage piloté par les quantités vendues.

# Les ventes sont fortement influencées par le prix,
# tandis que le stock semble principalement ajusté en fonction des volumes de vente plutôt que du niveau de prix.
```

Etape 5.6 - Mise à disposition de la nouvelle table sur un fichier Excel

```
In [218... #Mettre le dataset df_merge sur un fichier Excel
#Cette étape peut être utile pour partager le résultat du dataset obtenu avec les équipes.

df_base_finale.to_excel("export_base_excel.xlsx", index=False)
```

```
In [109... !jupyter nbconvert --to html --embed-images "/content/Secqueville_Thomas_1_notebook_012026.ipynb"

[NbConvertApp] Converting notebook /content/Secqueville_Thomas_1_notebook_012026.ipynb to html
[NbConvertApp] WARNING | Alternative text is missing on 1 image(s).
[NbConvertApp] Writing 874779 bytes to /content/Secqueville_Thomas_1_notebook_012026.html
```